

CEB-GNRD OpenBMC 快速上手与开发指南 (Quick Start Guide)

目录

参考资料与推荐教程

一、OpenBMC 开发与分层规范原则

二、构建环境与完整依赖准备

1. 安装 OpenBMC / Yocto / QEMU 全套依赖
2. 设置 UTF-8 语言环境

三、从零开始构建 ceb-gnrd

1. 获取并初始化源码仓库
2. 初始化构建环境与配置
3. 构建 OpenBMC 镜像与关键目标
4. 构建产物说明

四、在 QEMU 虚拟机中启动与测试

0. 推荐方式：用 run-qemu.sh 启动带硬件模型的模拟器
1. 备用方式：直接用 qemu-system-arm 启动
2. 访问 OpenBMC 服务
3. 系统服务状态检查
4. U-Boot 的 TFTP 使用（虚拟机与物理主板的区别）

五、Devtool 常用操作与板级开发全流程

devtool 核心开发流速览

1. 修改现有开源组件并生成板级 Patch
2. 向现有组件中增加全新源代码文件 / 模块
3. 使用 devtool add 创建全新的独立软件包 / 配方
4. devtool update-recipe 与 devtool finish 的核心区别与配合
5. devtool 其他实用命令速查

六、当前功能与实现状态

1. 板级硬件与固件布局
2. GPIO 行为
3. Web 界面
4. IPMI

七、验证清单

0. 一键检查脚本 ceb-gnrd-check
1. 构建后先在 QEMU 里检查
2. 上板后重点验证（对应端口指南红字项）

八、构建加速与常见问题

1. 缩短编译时间
2. 本项目遇到过的典型错误
3. 修改网页补丁的建议流程

九、板卡层实现说明 (原 meta-ceb-gnrd/README.md)

平台功能

待确认事项

IPMI 里的板卡版本

十、代码目录与文件清单 (meta-ceb-gnrd)

10.1 目录总览

10.2 conf/

10.3 recipes-bsp/u-boot/

10.4 recipes-kernel/linux/

10.5 recipes-core/、 recipes-extended/、 recipes-connectivity/、 recipes-devtools/

10.6 recipes-x86/chassis/ (主机电源控制)

10.7 recipes-phosphor/buttons/、 console/、 dump/、 logging/、 images/、 packagegroups/、 initrdscripts/

10.8 recipes-phosphor/configuration/ (配置与硬件契约)

10.9 recipes-phosphor/fans/

10.10 recipes-phosphor/flash/、 fru/、 health/

10.11 recipes-phosphor/interfaces/ (bmcweb)

10.12 recipes-phosphor/ipmi/

10.13 recipes-phosphor/leds/、 network/、 psu/、 rtc/

10.14 recipes-phosphor/sel-logger/、 sensors/、 settings/、 state/、 utils/、 watchdog/

10.15 recipes-phosphor/webui/

10.16 tools/ (开发辅助, 不进入镜像)

本文档详细说明如何从零开始构建 `ceb-gnrd` 固件镜像，如何在 QEMU 模拟器中启动并测试 WebUI/SSH/Redfish，以及如何遵循 OpenBMC 开发规范、使用 `devtool` 进行源码修改、新增代码与板级 Patch 产出。

硬件平台说明： 本项目面向 Intel Xeon 6 单路主板平台，BMC 芯片为 **ASPEED AST2600**。QEMU 模拟环境主要用于验证 OpenBMC 用户空间、`bmcweb`、WebUI 前端、SSH、网络配置及核心 D-Bus 接口流程，不能替代真实硬件环境中的 eSPI、PECI、ADC、PMBus、PWM/TACH 和实体网口测试。

参考资料与推荐教程

- 官方/社区核心教程：[Home | OpenBMC Guide Tutorial](#)
- OpenBMC 官方文档：[OpenBMC Documentation](#)
- Yocto Devtool 官方参考手册：[Yocto Application Development and the Extensible SDK](#)
- OpenBMC WebUI 前端仓库：[webui-vue](#)

一、OpenBMC 开发与分层规范原则

在 OpenBMC 开发中，严格遵循分层隔离与代码收敛原则，能确保后续平滑同步社区上游更新：

1. 板级目录隔离原则（Board-Level Layering）：

- 严禁直接修改上游图层（如 `upstream-layers/openembedded-core`、`meta-phosphor`、`meta-aspeed`、`meta-openembedded` 等）。
- 所有针对本板卡的修改（设备树 DTS、Entity-Manager JSON、IPMI YAML、systemd 网卡配置、自定义服务脚本、上游软件功能补丁）**必须全部集中收敛在板卡级目录**：`meta-ctopai/meta-ceb-gnrd/`。

1. Patch 与 bbappend 管理规范：

- 对任何开源组件（如 Linux 内核、`bmcweb`、`webui-vue`、`dbus-sensors` 等）的修改与新增代码，均应通过**板卡层中的 `.bbappend` + `.patch` 补丁**方式引入。
- 补丁文件存放在对应配方目录的 `files/` 下，使用 `SRC_URI:append` 引入。
- 补丁必须具备清晰的 Git Commit Title、功能说明和 `Signed-off-by` 签名。

1. 配置与接口解耦（D-Bus / Redfish 标准化）：

- 硬件拓扑与传感器接入优先通过 Entity-Manager JSON 配置，利用 `dbus-sensors` 自动加载，避免在底层驱动或服务中硬编码物理 I2C 地址。

二、构建环境与完整依赖准备

以下命令应在 Linux 主机或虚拟机中执行（推荐使用 **Ubuntu 22.04 LTS / 24.04 LTS**）。

1. 安装 OpenBMC / Yocto / QEMU 全套依赖

原生的 Yocto 基础包仅涵盖通用编译工具，进行 OpenBMC 开发、测试和模拟器验证还需要补充 **QEMU ARM 模拟器**、**IPMI 测试工具**、**Redfish/JSON 解析工具** 以及 **Python YAML/打包库**。

请直接运行以下一键安装命令：

```
sudo apt update
sudo apt install -y \
    build-essential chrpath diffstat gawk git wget curl \
    libegl1-mesa libsdl1.2-dev python3 python3-git \
    python3-jinja2 python3-pexpect python3-subunit python3-pip \
    python3-yaml python3-setuptools socat texinfo unzip xterm \
    zstd lz4 cpio file locales libssl-dev jq ipmitool \
    qemu-system-arm qemu-utils openssh-client
```

关键依赖分类说明：

- **Yocto / BitBake 核心构建依赖：** build-essential、diffstat、gawk、chrpath、texinfo、zstd、lz4、cpio、file、socat、unzip
- **Python 开发与 YAML 解析库：** python3-pip、python3-yaml（处理 IPMI/Entity-Manager YAML 配置）、python3-jinja2、python3-git
- **QEMU 模拟与固件运行工具：** qemu-system-arm（运行 AST2600 模拟器必选）、qemu-utils
- **BMC 联调与测试工具：** ipmitool（执行 IPMI 指令/读写 FRU/传感器）、curl & jq（测试与格式化 Redfish API 返回的 JSON 数据）、openssh-client（SSH 登录）

2. 设置 UTF-8 语言环境

```
sudo locale-gen en_US.UTF-8
export LANG=en_US.UTF-8
export LC_ALL=en_US.UTF-8
```

三、从零开始构建 ceb-gnrd

1. 获取并初始化源码仓库

```
git clone https://github.com/flether-hub/openbmc.git openbmc
cd openbmc
git submodule update --init --recursive
```

确认板级层结构完整：

```
test -f meta-ctopai/meta-ceb-gnrd/conf/machine/ceb-gnrd.conf && \
    echo "ceb-gnrd machine layer found"
```

2. 初始化构建环境与配置

在 OpenBMC 开发中，通常有两种初始化环境的方式：

方式 A：使用 `. setup ceb-gnrd`（推荐：用于全新创建与初始化）

```
cd ~/openbmc
. setup ceb-gnrd
```

- **作用：**OpenBMC 定制的引导配置脚本。它会自动创建 `build/ceb-gnrd` 目录，生成包含正确板卡配置的 `conf/local.conf`（自动注入 `MACHINE = "ceb-gnrd"`）以及完整的 `conf/bblayers.conf` 图层配置，并自动切换当前 Shell 至构建目录。
- **适用场景：**首次构建、全新创建构建目录或需要重新生成标准构建配置时使用。

方式 B：使用 `source oe-init-build-env build/ceb-gnrd`（用于日常进入已有环境）

```
cd ~/openbmc
source oe-init-build-env build/ceb-gnrd
```

- **作用：**OpenEmbedded / Yocto 原生环境加载脚本。它主要负责配置当前 Shell 的 `PATH`、`BUILDDIR` 等 BitBake 运行时环境变量，并将工作路径切入 `build/ceb-gnrd`。它直接复用已有的配置文件，不会重新覆写。
- **适用场景：**日常开发、新开终端窗口或重新登录后继续编译已有构建目录时使用。

💡 两者核心区别对比速查

对比项	<code>. setup ceb-gnrd</code>	<code>source oe-init-build-env build/ceb-gnrd</code>
功能定位	OpenBMC 定制的一键初始化脚本	Yocto 原生环境变量配置脚本

对比项	. setup ceb-gnrd	source oe-init-build-env build/ceb-gnrd
配置文件生成	自动生成/覆写 local.conf 与 bblayers.conf	不生成新配置，直接复用现有配置文件
MACHINE 设定	自动在配置中锁定目标板卡为 ceb-gnrd	依赖已有配置或需手动声明 export MACHINE=...
主要使用时机	首次初始化 / 全新构建时使用	日常开发 / 新窗口恢复环境时使用

3. 构建 OpenBMC 镜像与关键目标

- 构建完整 BMC SPI Flash 固件镜像：

```
bitbake obmc-phosphor-image
```

- 单独编译 Linux 内核与设备树 (DTS)：

```
bitbake virtual/kernel
```

- 单独编译与验证板级配置包：

```
bitbake entity-manager dbus-sensors ceb-gnrd-hardware-contract
```

4. 构建产物说明

构建产物存放于： build/ceb-gnrd/tmp/deploy/images/ceb-gnrd/

- obmc-phosphor-image-ceb-gnrd-*.static.mtd：BMC SPI 原始固件镜像（供编程器烧录或 QEMU 启动）。
- obmc-phosphor-image-ceb-gnrd-*.tar.gz：BMC 根文件系统压缩包。
- fitImage：包含 Linux 内核与 Initramfs 的镜像。
- aspeed-ceb-gnrd.dtb：编译后的板级设备树二进制。

四、在 QEMU 虚拟机中启动与测试

0. 推荐方式：用 run-qemu.sh 启动带硬件模型的模拟器

bitbake obmc-phosphor-image 完成后，在构建机上直接运行（仓库根目录有一个指向 meta-ctopai/meta-ceb-gnrd/tools/qemu/run-qemu.sh 的符号链接）：

```
cd ~/openbmc
./run-qemu.sh
```

脚本使用镜像 `qemuboot.conf` 指向的 BitBake native QEMU (已带本板的模型补丁, 见 `tools/qemu/patches/`), 并按真实地址挂上: BMC Flash 与 64 MiB BIOS Flash、`eth0` (MAC2) 与 `eth1` (NC-SI)、4 个温度传感器 (`i2c-6` 的 `0x48-0x4b`)、PSU 槽位 (`i2c-7` 的 `0x58-0x5a`)、FRU EEPROM (`i2c-10`, 1 KiB)、RTC (NCT3015Y, `i2c-9` 的 `0x6f`)、风扇转速、ADC、PECI CPU, 以及一个模拟主机 (上电/复位时序、POST 码、COM1 串口、KCS、VGA、USB)。随后在 `http://127.0.0.1:8800` 打开浏览器控制面板, 可以按电源/UID 按钮、注入温度/电压/PSU/风扇故障、切换网络状态等。

服务	地址
BMC Web / Redfish	<code>https://127.0.0.1:8443</code>
BMC SSH	<code>ssh -p 2222 root@127.0.0.1</code>
IPMI LAN (UDP)	<code>127.0.0.1:2623</code>
硬件模拟控制台	<code>http://127.0.0.1:8800</code>

常用环境变量: `DEPLOY` (镜像目录)、`STATE` (状态目录, 默认 `~/qemu-ceb-gnrd`, 存放 FRU 文件、日志、QMP socket)、`BIOS_FLASH`、`PANEL_PORT`、`NO_PANEL=1`、`NETWORK_CAPTURE=1`、`PECI_CPU=gnrd|spr`。模拟器的详细说明、接口范围和已知限制见 `tools/qemu/README.md`。下面的小节是不带硬件模型的“裸 QEMU”写法, 只用于验证 Web/SSH/Redfish 等用户空间功能。

1. 备用方式: 直接用 `qemu-system-arm` 启动

CEB-GNRD 设备树禁用了 MAC0 和 MAC3, 管理口 `eth0` 使用 **MAC2 (设备树标签 `&mac1`, 对应 QEMU 的第 2 个网卡)**, `eth1` (NC-SI) 使用 MAC3 (`&mac2`)。所以 QEMU 需要建两个网卡: 第 1 个只是占位, 第 2 个才是 `eth0`, 并让 QEMU 的用户网络网段为 `192.168.185.0/24`、DHCP 地址池从 `192.168.185.200` 开始 (`dhcpstart=`): `eth0` 默认用 DHCP 取地址, 第一个租约正好是 `192.168.185.200`, 端口转发才能找到它:

```
cd ~/openbmc/build/ceb-gnrd/tmp/deploy/images/ceb-gnrd
qemu-system-arm -M ast2600-evb -m 1G -nographic -monitor none \
  -drive file=obmc-phosphor-image-ceb-gnrd.static.mtd,format=raw,if=mtd \
  -nic user \
  -nic user,net=192.168.185.0/24,host=192.168.185.1,dhcpstart=192.168.185.200,hostfwd=tcp:127.0.0.1:8443-192.168.185.200:443,hostfwd=tcp:127.0.0.1:2222-192.168.185.200:22,hostfwd=udp:127.0.0.1:2623-192.168.185.200:623
```

- 串口控制台就是调试口 UART5 (`ttys4`), 日志直接显示在当前终端; 退出 QEMU: 先按 `Ctrl-A`, 再按 `x`。
- `ttys4` 的登录提示等待 `multi-user.target` 和 `obmc-led-group-start@bmc_booted.service` 启动任务结束后显示。服务任务失败也不会禁止登录; 不等待所有服务健康。启动任务卡住可能延迟登录, 登录后后台日志仍可能输出。该实例移除了 `Before=getty.target` 并关闭隐式目标排序, 显式保留系统初始化和关机依赖, 避免启动顺序循环。
- 想后台运行可放进 `tmux`: `Ctrl-B` 再按 `D` 暂离, `tmux attach` 回来。
- 把命令保存成脚本更方便: `~/run-bmc.sh`。

QEMU 里 ping 的限制：QEMU 的 `user` 网络 (slirp) 是虚拟的 NAT 网络。① 它默认不转发 ICMP，ping 外网大多不通，但 TCP/UDP (`curl`、`nslookup`) 是通的，验证外网请用 `curl` 而不是 `ping`；② 192.168.185.0/24 整个网段都在 QEMU 内部，宿主机的真实地址 (如 192.168.185.84) 在里面是不存在的，虚拟机访问宿主机用 `host=` 指定的 192.168.185.1，DNS 是 192.168.185.3。

U-Boot 启动方式与 TFTP：U-Boot 固定从本地 SPI 闪存启动，TFTP 只用于手动网络启动调试和 `run netupdate` 更新闪存；QEMU 里的服务器地址和物理板子不同 (192.168.185.1，需要 `tftp=` 参数)，详见本章 “4. U-Boot 的 TFTP 使用”。

⚠ **待验证：**上面的网卡对应关系按 QEMU 的网卡分配规则推断，网页打不开时先在 BMC 控制台里看 `ip addr show eth0` 是否有 192.168.185.200。如果你之前用的是旧版三网卡写法 (`-net nic -net nic -net nic,netdev=net0`)，那条命令对应的是旧的 MAC 映射，请改用本节的写法。

2. 访问 OpenBMC 服务

- **Web 管理界面 (HTTPS)：**浏览器打开：`https://127.0.0.1:8443/#/`

浏览器证书警告绕过：Chrome/Edge 提示 “您的连接不是私密连接” 时，在键盘上直接盲打输入 `thisisunsafe` 即可进入登录页面。

- **登录账号：**用户名 `root`，密码 `0penBmc` (`allow-root-login` 已启用)。
- **SSH 登录：**

```
ssh -p 2222 root@127.0.0.1
```

- **Redfish API 接口测试** (在宿主机 Ubuntu 终端里执行；BMC 镜像里也带了 `curl`，在 BMC 里用 `https://127.0.0.1/` 不带端口)：

```
curl -k https://127.0.0.1:8443/redfish/v1/
# 以下需要登录；-u 用户名:密码，-k 忽略自签名证书
curl -k -u root:0penBmc https://127.0.0.1:8443/redfish/v1/Systems/system
curl -k -u root:0penBmc https://127.0.0.1:8443/redfish/v1/Chassis/chassis/Sensors      #
传感器列表
curl -k -u root:0penBmc https://127.0.0.1:8443/redfish/v1/Systems/system/LogServices/EventLog/Entries  # 事件日志 (SEL)
curl -k -u root:0penBmc https://127.0.0.1:8443/redfish/v1/Managers/bmc/LogServices/Dump/Entries  # 转储列表
```

创建一个 BMC 转储（返回一个任务，QEMU 里约 30 秒完成，期间再提交会返回 503 ResourceInUse ）：

```
curl -k -i -u root:openBmc -X POST -H "Content-Type: application/json" \
  -d '{"DiagnosticDataType":"Manager"}' \
  https://127.0.0.1:8443/redfish/v1/Managers/bmc/LogServices/Dump/Actions/LogService.CollectDiagnosticData
curl -k -u root:openBmc https://127.0.0.1:8443/redfish/v1/TaskService/Tasks/0      # 看任务状态
```

-i 会把 HTTP 状态码和响应头也打印出来，排查接口错误时很有用。

3. 系统服务状态检查

在 BMC 终端中运行：

```
# 检查失败服务
systemctl --failed

# 检查 Web 后端与 Socket 监听
systemctl status bmcweb
systemctl status bmcweb.socket

# 检查实体管理服务与 IP 状态
systemctl status xyz.openbmc_project.EntityManager.service
ip addr
```

4. U-Boot 的 TFTP 使用（虚拟机与物理主板的区别）

启动方式：U-Boot 的默认 bootcmd 固定为 run bootspi，只从本地 SPI 闪存加载内核，不会自动走网络。TFTP 只在你手动操作时使用：手动网络启动（调试内核用）和 run netupdate（更新闪存，见 4.5）。

项目	物理主板	QEMU 虚拟机
TFTP 服务器地址	192.168.185.84 (Ubuntu 主机, U-Boot 默认 serverip)	192.168.185.1 (QEMU 虚拟网络里的“宿主机”，host= 指定)
TFTP 服务由谁提供	Ubuntu 上的 tftpd-hpa (目录 /srv/tftp)	QEMU 自带，启动命令里加 tftp=/srv/tftp
网络	必须和 Ubuntu 在同一局域网（经交换机），192.168.185.0/24	QEMU 的 -nic user 虚拟网络，只在虚拟机内部有效
手动 TFTP 前要做什么	无 (serverip 默认就是 .84)	手动 setenv serverip 192.168.185.1
mii、PHY 寄存器、RGMII 时序	真实硬件，必须上板验证	模拟的（所有 PHY 地址都应答，页寄存器读出全 0），不能代表真实板子

镜像： `fitImage` 只含内核、设备树和 `initramfs`，手动网络启动只替换这一部分，根文件系统仍用本地闪存的 `rofs`。文件在 `build/ceb-gnrd/tmp/deploy/images/ceb-gnrd/fitImage`，文件名必须保持 `fitImage`。

4.1 在 Ubuntu 上准备 TFTP 服务（物理主板和 QEMU 都用同一个目录）

```
sudo apt install -y tftpd-hpa
sudo tee /etc/default/tftpd-hpa >/dev/null <<'EOF'
TFTP_USERNAME="tftp"
TFTP_DIRECTORY="/srv/tftp"
TFTP_ADDRESS="0.0.0.0:69"
TFTP_OPTIONS="--secure --create"
EOF
sudo mkdir -p /srv/tftp
sudo cp ~/openbmc/build/ceb-gnrd/tmp/deploy/images/ceb-gnrd/fitImage /srv/tftp/fitImage
sudo chmod 644 /srv/tftp/fitImage
sudo chown -R tftp:tftp /srv/tftp
sudo systemctl restart tftpd-hpa
# 自测：能下载下来且大小和源文件一致
cd /tmp && tftp 127.0.0.1 -c get fitImage && ls -l fitImage
```

每次重新编译后，要重新复制 `fitImage`。

4.2 QEMU 虚拟机里手动网络启动

QEMU 启动命令里给第二个网卡加 `tftp=/srv/tftp`：

```
cd ~/openbmc/build/ceb-gnrd/tmp/deploy/images/ceb-gnrd
qemu-system-arm -M ast2600-evb -m 1G -nographic -monitor none \
  -drive file=openbmc-phosphor-image-ceb-gnrd.static.mtd,format=raw,if=mtd \
  -nic user \
  -nic user,net=192.168.185.0/24,host=192.168.185.1,dhcpstart=192.168.185.200,tftp=/srv/tftp,hostfwd=tcp:127.0.0.1:8443-192.168.185.200:443,hostfwd=tcp:127.0.0.1:2222-192.168.185.200:22,hostfwd=udp:127.0.0.1:2623-192.168.185.200:623
```

看到 `Hit any key to stop autoboot` 时按任意键，在 `ast#` 提示符下：

```
setenv serverip 192.168.185.1
tftpboot 0x83000000 fitImage
bootm 0x83000000
```

成功的标志：`TFTP from server 192.168.185.1`，一排 `#`，`Bytes transferred = 5656980`（字节数应与 `fitImage` 大小一致），然后 `## Loading kernel from FIT Image at 83000000` 并进入 Linux。`/srv/tftp/fitImage` 要对运行 QEMU 的用户可读。

在 QEMU 里不改 `serverip` (还是 192.168.185.84) 直接 `tftpboot` , 会停在 `Loading: *` , 最后 `ARP Retry count exceeded` , 因为 .84 在虚拟网络里不存在。

4.3 物理主板上手动网络启动

1. Ubuntu 的网口配好 192.168.185.84/24 , BMC 的网口和它在同一局域网 (经交换机或直连) 。
2. 烧入新编译的 U-Boot 和镜像后, **第一次要重置 U-Boot 环境** (环境存在闪存里; 如果以前保存过带 `tftpboot` 的旧 `bootcmd` , 会盖住新默认值, 板子仍会先走网络) , 并按需设置 MAC。U-Boot 默认带有固定的本地管理地址 (`ethaddr=02:26:00:00:00:01` 、 `eth1addr=02:26:00:00:00:02`) , 已保存的环境里缺这两项时, 启动时会自动补齐并保存; 真实板卡应写入每板唯一的生产 MAC:

```
env default -a
setenv ethaddr 02:00:00:00:00:01
saveenv
reset
```

`env default -a` 不可用时, 手动设置: `setenv bootcmd 'run bootspi'; setenv ipaddr 192.168.185.200; setenv netmask 255.255.255.0; setenv gatewayip 192.168.185.1; setenv serverip 192.168.185.84; saveenv` 。

1. 重启后应直接从本地闪存启动, 不出现 TFTP。要手动网络启动时, 在 `Hit any key to stop autoboot` 时按任意键, 然后执行 `tftpboot 0x83000000 fitImage` 和 `bootm 0x83000000` , 串口日志里应出现 `TFTP from server 192.168.185.84; our IP address is 192.168.185.200` 和 `Bytes transferred` 。
2. 出问题时的排查 (只适用于物理板子) :
 - Ubuntu: `ip -br addr` (地址在接 BMC 的网卡上) 、 `sudo ufw status` (放开 UDP 69) 、 `sudo journalctl -u tftpd-hpa -f` (看有没有收到请求) 。
 - 抓包 (局域网流量大, 用 BMC 的 MAC 过滤) : `sudo tcpdump -i <网卡> -n -e ether host 02:00:00:00:00:01` 。
 - 链路已 up 但 ARP/TFTP 不通: 用 `sudo ethtool -s <网卡> speed 100 duplex full autoneg on` 把链路限制到 100M 再试; 100M 通而 1000M 不通, 说明是 RGMII 延时问题, 可在 U-Boot 里用 `mii write 2 0x1f 0xd08` 后读写 `0x11` (TX 延时, bit 8) 和 `0x15` (RX 延时, bit 3) 做实验, 确认后再改设备树的 `phy-mode` 。

4.4 U-Boot 里的 NC-SI 口

NC-SI 口 (&mac2) 在 U-Boot 里**不启动** (设备树里禁用) : U-Boot 只用 eth0 取镜像。不要给它加 `phy-mode` 再启用, 那样 NC-SI 探测会让 U-Boot 崩溃 (`data abort` , 不断复位) ; 不加则只会打印 `Invalid PHY interface '<NULL>'` 。Linux 里 NC-SI 口 `eth1` 自动使能并用 DHCP 取地址: 主机上电后由 `ceb-gnrd-ncsi` 服务把 `eth1` 拉起 (E810 没有待机供电, 主机关机时 NC-SI 不可用, 拉起后内核 NC-SI 栈自动选择通道; E810 上电后不一定立刻就绪, 服务会在 30 秒内没有链路时对 `eth1` 做 down/up 重试, 每 30 秒一次, 最多 3 次, 日志见 `journalctl -t ceb-gnrd-ncsi`) , `systemd-networkd` 按 `DHCP=ipv4` 获取地址。

4.5 在 U-Boot 里通过 TFTP 更新闪存 (`run netupdate`)

U-Boot 默认环境里带了变量 `netupdate`，把“TFTP 取内核和根文件系统，再写进 SPI 闪存”合成一条命令：

```
run netupdate
reset
```

它做的事（任何一步失败都会停止并打印 `Network update FAILED`）：

1. `sf probe 0`，选中 BMC 的 SPI 闪存。
2. 从 `serverip` 取 `image-kernel`（变量 `netupdate_kernel`）到内存 `0x90000000`，检查大小不超过 9 MiB，`sf update` 写到 `0x100000`。
3. 取 `image-rofs`（变量 `netupdate_rofs`），检查大小不超过 44 MiB，`sf update` 写到 `0xa00000`。
4. 成功后提示 `Network update done, run reset`。

不会改动：U-Boot 本体和环境（`0x000000` 到 `0x0fffff`，所以保存的 `ethaddr`、`bootcmd` 不丢）、可写分区 `rwfs`（`0x3600000`，所以用户配置不丢）。

准备：把 `image-kernel`、`image-rofs` 放进 TFTP 目录（`sudo cp -L <deploy 目录>/image-kernel /srv/tftp/` 和 `image-rofs`，`chmod 644`）。物理主板上 `serverip` 默认就是 192.168.185.84；QEMU 里先 `setenv serverip 192.168.185.1`（并且 QEMU 要带 `tftp=`）。文件名不同时，用 `setenv netupdate_kernel <名字>` / `setenv netupdate_rofs <名字>` 修改。

注意：

- 写入过程中断电可能让镜像损坏；这条命令不写 U-Boot，所以即使失败，也还能进入 U-Boot 重新执行。
- 旧的已保存环境会盖住新默认值，第一次需要 `env default -a; saveenv`（见 4.3）。
- 在 QEMU 里它会改写 `.mtd` 文件，先备份：`cp obmc-phosphor-image-ceb-gnrd.static.mtd backup.mtd`。
- `0x90000000` 是内存里的空闲区，1008 MiB 内存足够；分区偏移和大小来自设备树，改了分区布局要同步修改 `ceb-gnrd-env.h`。

五、Devtool 常用操作与板级开发全流程

`devtool` 是 Yocto 推荐的源码级开发与补丁管理利器。使用 `devtool` 可以修改现有组件、向现有组件新增源代码文件、创建全新的独立软件包，并一键归档存入指定的板卡级图层。

💡 devtool 核心开发流速览

开发场景	核心 workflow 步骤
场景 A: 修改或向现有组件新增代码	<code>devtool modify <recipe></code> → 在 <code>workspace</code> 中修改/新增文件并 <code>git commit</code> → <code>devtool build <recipe></code> → <code>devtool finish <recipe> ../meta-ctopai/meta-ceb-gnrd</code> (自动生成 Patch 并追加进板卡层)
场景 B: 创建全新的独立软件包/服务	<code>devtool add <pkg> <src_path/url></code> → 自动生成配方并在 <code>workspace</code> 调试构建 → <code>devtool finish <pkg> ../meta-ctopai/meta-ceb-gnrd</code> (自动将新配方归档至板卡层)

1. 修改现有开源组件并生成板级 Patch

以修改 `webui-vue` 前端界面为例：

(1) 提取源码到工作区 (`devtool modify`)

```
cd ~/openbmc/build/ceb-gnrd
devtool modify webui-vue
```

Yocto 会自动将 `webui-vue` 源码提取到 `build/ceb-gnrd/workspace/sources/webui-vue/` 并初始化为 Git 仓库。

(2) 修改代码并提交

```
cd workspace/sources/webui-vue
# 进行代码修改 ...

# 检查修改状态并提交
git status
git diff
git add .
git commit -m "ceb-gnrd: Customize webui theme

Signed-off-by: Your Name <your.email@example.com>"
```

(3) 实时编译与验证

```
cd ~/openbmc/build/ceb-gnrd
devtool build webui-vue

# 或构建完整镜像并在 QEMU 中验证
devtool build-image obmc-phosphor-image
```

(4) 生成 Patch 并归档至板级目录 (`devtool finish`)

```
devtool finish webui-vue ../meta-ctopai/meta-ceb-gnrd
```

- 执行结果:

1. 自动在 `meta-ctopai/meta-ceb-gnrd/recipes-phosphor/webui/files/` 目录下生成标准的 `000x-ceb-gnrd-*.patch`。
2. 自动在 `meta-ctopai/meta-ceb-gnrd/recipes-phosphor/webui/webui-vue_%.bbappend` 的 `SRC_URI` 中追加该补丁引用。
3. 自动从 `workspace` 中清理开发状态 (无需再手动执行 `reset`)。

2. 向现有组件中增加全新源代码文件 / 模块

当需要为现有项目 (如 `bmcweb` 或 `dbus-sensors`) 新增自定义 `.cpp`、`.hpp` 或网页组件时:

(1) 进入源码工作区新增文件

```
cd ~/openbmc/build/ceb-gnrd
devtool modify dbus-sensors
cd workspace/sources/dbus-sensors
```

(2) 创建新文件并注册构建系统

```
# 创建新的传感器实现文件
touch src/CebGnrdSpecialSensor.cpp
touch src/CebGnrdSpecialSensor.hpp

# 在 CMakeLists.txt 或 meson.build 中将新文件加入编译列表
```

(3) 将新文件纳入 Git 管理并提交

```
git add src/CebGnrdSpecialSensor.cpp src/CebGnrdSpecialSensor.hpp CMakeLists.txt
git commit -m "ceb-gnrd: Add special sensor monitoring module"
```

```
Signed-off-by: Your Name <your.email@example.com>
```

(4) 编译并输出至板卡层

```
cd ~/openbmc/build/ceb-gnrd
devtool build dbus-sensors
devtool finish dbus-sensors ../meta-ctopai/meta-ceb-gnrd
```

devtool 会将包含新增代码文件的 *Git Commit* 完整转换为包含新建文件的 *Git Patch*，并自动保存到板卡级目录的 *files/* 中。

3. 使用 `devtool add` 创建全新的独立软件包 / 配方

当需要为 `ceb-gnrd` 引入一个全新的板级私有工具、自研服务或第三方库时：

(1) 基于本地源码或远程仓库新建配方

- 方式 A：基于本地源码目录：

```
cd ~/openbmc/build/ceb-gnrd
devtool add ceb-gnrd-tool /home/test/src/ceb-gnrd-tool
```

- 方式 B：基于远程 *Git* 仓库：

```
devtool add my-custom-app https://github.com/my-org/my-custom-app.git
```

devtool 会自动检测源码结构（如 *CMake*、*Meson*、*Autotools*、*Python*、*Makefile*），并在 `workspace/recipes/` 自动生成对应的 `.bb` 配方文件。

(2) 编译与调试配方

```
devtool build ceb-gnrd-tool
```

如果需要微调配方逻辑（如追加依赖、修改安装路径），可直接编辑 `workspace/recipes/ceb-gnrd-tool/ceb-gnrd-tool.bb`。

(3) 归档全新配方到板卡层

```
devtool finish ceb-gnrd-tool ../meta-ctopai/meta-ceb-gnrd
```

执行后，该新配方会自动安装在 `meta-ctopai/meta-ceb-gnrd/recipes-ceb-gnrd/ceb-gnrd-tool/ceb-gnrd-tool.bb`。

(4) 将新软件包打包加入板卡镜像

在 `meta-ctopai/meta-ceb-gnrd/recipes-phosphor/packagegroups/packagegroup-ceb-gnrd-apps.bb` 中加入：

```
RDEPENDS:${PN}-system += " \
    ceb-gnrd-tool \
    "
```

或在 `meta-ctopai/meta-ceb-gnrd/recipes-phosphor/images/obmc-phosphor-image.bbappend` 中追加：

```
IMAGE_INSTALL:append = " ceb-gnrd-tool"
```

4. `devtool update-recipe` 与 `devtool finish` 的核心区别与配合

对比维度	devtool update-recipe	devtool finish
一句话定位	阶段性更新（保留开发状态）	最终交付并退出（完成开发）
生成 Patch / 更新配方	✅ 会将 Git 提交生成 Patch 并更新至指定图层	✅ 会将 Git 提交生成 Patch 并更新至指定图层
工作区状态（workspace）	🟢 继续保留，处于开发模式	🔴 自动清理并删除 workspace 中的源码和临时配方
后续 bitbake 编译来源	依然优先使用 workspace/sources/ 下的代码	回归标准流程（从上游下载解压 + 板卡层 Patch 打补丁）
适用场景	边开发边同步 Patch、需要继续调试修改	功能完全开发、测试通过，准备提交代码归档

⚠️ 常见疑问：`devtool finish` 执行完后还需要执行 `devtool reset` 吗？

答案：完全不需要！因为 `devtool finish` 的底层逻辑已经自动包含了 `reset` 清理机制。执行成功后，`build/ceb-gnrd/workspace/sources/<recipe>` 目录和临时配置已被自动清除，该组件已彻底退出开发模式。如果此时再执行 `devtool reset`，系统会报错提示该配方不在工作区中。

只有在主动放弃当前修改、中途丢弃所有未保存改动时，才需要手动执行 `devtool reset <recipe>`（若有未提交的改动可加 `-f` 强制放弃：`devtool reset -f <recipe>`）。

💡 推荐协同工作流：

```
# 1. 提取源码进入开发模式
devtool modify bmcweb

# 2. 修改代码并提交
cd workspace/sources/bmcweb
git commit -am "ceb-gnrd: Add custom redfish OEM handler"

# 3. 编译验证
devtool build bmcweb

# 4. [阶段性保存] 临时同步 Patch 到板卡层，不退出开发（必须加 -a 指定板卡层）
devtool update-recipe -a ../meta-ctopai/meta-ceb-gnrd bmcweb

# 5. [最终交付] 全部功能测试通过后，归档 Patch 并彻底清理工作区（会自动清理 workspace）
devtool finish bmcweb ../meta-ctopai/meta-ceb-gnrd
```

5. devtool 其他实用命令速查

操作需求	执行命令
仅更新补丁不退出工作区	devtool update-recipe -a ../meta-ctopai/meta-ceb-gnrd <recipe>
构建包含工作区代码的完整镜像	devtool build-image obmc-phosphor-image
放弃修改/重置工作区	devtool reset <recipe> （强制放弃加 -f ）
查看当前处于工作区的配方列表	devtool status

六、当前功能与实现状态

下表只描述代码里已经实现的内容；标 ⚠️ 的项还没有上板验证或存在疑问，详见 port_guide.xlsx 底部红字的“待澄清”区块。

1. 板级硬件与固件布局

项目	现状
BMC Flash	W25Q512JV 64 MiB; 布局: U-Boot / 环境变量 / 内核 9 MiB / ROFS 44 MiB / RWFS 10 MiB (FLASH_RWFS_OFFSET:flash-65536 = "55296" , 设备树分区与之对应)
管理网口 eth0	MAC2 + RTL8211FS (rgmii , PHY 地址 2 ⚠️ , 复位由 CPLD 控制) , 默认 IPv4 DHCP (地址、网关、DNS 由 DHCP 服务器提供; run-qemu.sh 的模拟器里地址池从 192.168.185.200 开始)
NC-SI 网口 eth1	MAC3, Linux 里默认 DHCP; E810 没有待机供电, 主机上电后由 ceb-gnrd-ncsi 自动拉起 (30 秒内没有链路就 down/up 重试, 每 30 秒一次, 最多 3 次) , 主机关机时关闭; U-Boot 里不启动该口
MAC 地址	保存在 U-Boot 环境变量 ethaddr / eth1addr , 固件升级不会擦除 u-boot-env 分区
ADC	内部 2.5 V 参考电压; D3V0_BAT0 因 R542/Q39 未焊会饱和 ⚠️
eSPI	仅 Peripheral 通道; 驱动带复位恢复、错误计数和 debugfs 日志 ⚠️ (见下)
RTC / 时间	AST2600 内部 RTC (无电池) 已关闭, 板上 NCT3015Y 是 rtc0 ; 内核开机校时, ceb-gnrd-rtc-sync 兜底 (最多等 /dev/rtc0 3 秒, 启动超时 5 秒; 没有 /dev/rtc0 的环境, 例如不带板级补丁的裸 QEMU, 会等满 3 秒; run-qemu.sh 的模拟器里有 NCT3018Y 模型) ; BMC 系统时间和 SEL 时间默认来自 RTC
主机 KCS	设备树 &kcs3 带 aspeed,lpc-io-reg = <0xca2> (驱动必需) , 未配 SerIRQ; 需要 eSPI 外设通道就绪、BIOS 把 KCS 端口设为 0xCA2 才通 ⚠️
U-Boot	固定从本地 SPI 闪存启动 (bootcmd = run bootspi) ; 默认网络参数与 Linux eth0 一致 (用于手动 TFTP) ; run netupdate 通过 TFTP 更新闪存里的内核和 rofs (见第四章 4.5) ; NC-SI 口在 U-Boot 里禁用
PSU	ceb-gnrd-psu-detect 每 5 秒探测 0x58/0x59/0x5A, 仅为在位模块创建 pmbus 设备; 传感器有输入/输出电压、输入/输出功率和 PSUn_Temp (取 pmbus 的 temp2) ⚠️

项目	现状
SD / eMMC	本板没有，U-Boot 和 Linux 设备树都把 emmc、sdhci 相关节点设为 disabled，开机日志里不再有 mmc0: Failed to initialize a non-removable card
硬件看门狗	AST2600 WDT1 由 systemd 喂狗（RuntimeWatchdogSec=120s），复位类型 soc（只复位 SoC，不是整颗芯片，避免 GPIO 回到上电状态让 CPLD/主机侧信号抖动 ⚠️ 上板要确认复位期间这些电平稳定）。aspeed_wdt 不支持预超时，所以 systemd-conf 安装了一个和 meta-phosphor 同名的 /etc/systemd/system.conf.d/40-hardware-watchdog.conf（同名文件 /etc 里的覆盖 /usr/lib 里的），只保留 RuntimeWatchdogSec=120s 和 WatchdogDevice，去掉 RuntimeWatchdogPreSec、RuntimeWatchdogPreGovernor=panic，否则每次开机都会有 Failed to set watchdog pretimeout_governor 提示（曾试过用空赋值清除，systemd 报 Failed to parse RuntimeWatchdogPreSec=，不可行）。内核崩溃：CONFIG_PANIC_ON_OOPS + CONFIG_PANIC_TIMEOUT=5，5 秒后重启；打开了 Magic SysRq（echo c > /proc/sysrq-trigger 可以制造崩溃做测试，串口 BREAK 不能触发）。服务恢复（标准做法，ceb-gnrd-health）：drop-in 10-ceb-gnrd-restart.conf 给对象映射器、Entity-Manager、bmcweb、ipmid 和本层的风扇设置、温度最大值、告警灯服务设置 Restart=always、启动限制（5 分钟内 5 次）和 OnFailure=obmc-bmc-service-quiet@0.target，服务一直起不来时 phosphor-state-manager 把 BMC 置为 Quiesced，并按 phosphor-state-manager_%.bbappend 里打开的 auto-reboot-on-bmc-quiet 自动重启 BMC。上游没有次数限制，这里用 ceb-gnrd-quiet-reboot-limit.sh 限制为 最多自动重启 1 次 （计数存在读写分区，开机 15 分钟后由定时器清零）：重启后同一个故障还在，BMC 就停在 Quiesced 状态，不再自动重启，并写一条 SEL，需要人工处理；上游的守护进程不向 systemd 报活，所以只能恢复崩溃，进程还在但卡死不会被发现，本层自己的三个服务是 Type=notify + WatchdogSec=，主循环卡死会被重启；ceb-gnrd-wdt-reset-log 在没有干净关机标记而 bootstatus 显示看门狗复位时写 SEL（在 U-Boot 里 reset 也会被记一次）。以上新增部分没有编译、没有验证

2. GPIO 行为

GPIO	行为
UID 按键 BMC_UID_BUTTON_N_N (GPIOV0)	低有效，按下切换 identify 灯组；UID 灯 BMC_UID_LED (GPIOV1) 高有效
CPU 开关机 BMC_CPU_POWER_BUTTON (V2)、复位 BMC_CPU_RESET (V3)	低脉冲，由 x86-power-control 输出（200 ms / 强制关机 8 s / 复位 500 ms；强制关机脉冲在 power-config-host0.json 的 ForceOffPulseMs，bios-update.sh 里的 FORCE_OFF_PULSE_S 要比它大 1 秒）
BMC_CPU_POWER_DOWN (V4)	高有效输入，供状态机判断上电

GPIO	行为
电源按键输入 BMC_POWER_BUTTON_INPUT (GPIOM2)	只检测 ：按下时写 SEL 和日志，不触发开关机，也不直通到 CPU 电源按键输出
告警灯 BMC_SYS_ALERT_LED (GPIOI5)	内核 LED 名 <code>fault</code> ，由 <code>phosphor-led-manager</code> 的标准组 <code>enclosure_fault</code> 驱动（ <code>ceb-gnrd-alert-led</code> 只负责置位/清除该组的 <code>Asserted</code> ，每 30 秒重发一次以防 <code>led-manager</code> 重启丢状态）。电压越限点亮，温度到达 UNR（不可恢复上限）点亮； CPU_MAX_TEMP / DIMM_MAX_TEMP 到达 Upper Critical (98°C / 85°C) 也点亮 ，阈值告警解除且没有其他故障时熄灭；BIOS 启动超时（ 600 秒 ）点亮，后续 POST complete 成功后清除；watchdog 超时锁存点亮，BMC 重启后清除 ⚠️ 没有验证
BMC_FAN_BMC_OVERRIDE_N (GPIOI6)	风扇控制就绪后拉高，BMC 接管风扇；服务停止时拉低交还 CPLD。BMC 复位（看门狗或用户触发）期间 BMC 不能控制风扇，必须交还 CPLD，所以这根脚 不保持 ：内核对用户态申请的线都会置位 <code>reset tolerance</code> ， <code>ceb-gnrd-fan-owner</code> 在占住这根线后用 <code>devmem</code> 清掉 <code>0x1e7800ac</code> 的 bit6，复位时它回到输入态；清不掉就不接管风扇（留给 CPLD）。正常关机时服务停止， <code>ceb-gnrd-fan-release</code> 先把它拉低 ⚠️ 没有在板上验证（复位后这根脚悬空时 CPLD 是否接管风扇要确认）
复位保持 (reset tolerance)	AST2600 的 GPIO 每个引脚有 <code>reset tolerance</code> 位，置位的引脚在看门狗 SoC 复位时保持方向和输出值；用户态（ <code>x86-power-control</code> 、 <code>gpioset</code> ）申请一根线时内核自动置位。 <code>BMC_CPU_POWER_BUTTON</code> 、 <code>BMC_CPU_RESET</code> 、 <code>BMC_BIOS_FLASH_SELECT</code> 依赖这个行为，在用户触发的 BMC 复位和看门狗复位中保持状态； <code>BMC_BIOS_FLASH_SELECT</code> 只在 BIOS 升级时被改变，其他时间 BMC 不碰它 ⚠️ 没有在板上验证
BMC_HBLEED_N (GPIOP7)	eSPI 驱动就绪后启用内核 <code>heartbeat</code> 触发器
BMC_BIOS_FLASH_SELECT (GPIOM1)	BIOS 升级时拉高切给 BMC，等 5 秒后烧写，结束后拉低
BMC_BIOS_BOOT_OK (GPIOM7)	<code>x86-power-control</code> 的标准 <code>PostComplete</code> （高有效）：拉高时 D-Bus <code>xyz.openbmc_project.State.OperatingSystem</code> 的 <code>OperatingSystemState</code> 为 <code>Standby</code> ，否则为 <code>Inactive</code> （主机关机时也是 <code>Inactive</code> ）。主机开着时它的下降沿会进入 <code>x86-power-control</code> 的热复位检查并记一次软复位的重启原因， 不会产生电源脉冲 。告警灯服务读 <code>OperatingSystemState</code> 来取消当次 BIOS 启动超时告警 ⚠️ 没有在板上验证

3. Web 界面

- **已保留**：概要、事件日志、POST Code、转储、清单与 LED（系统/BMC/机箱三张表）、传感器、恢复出厂设置（仅 BMC）、KVM（含全屏）、固件、重启 BMC、SOL、服务器电源操作、虚拟媒体、日期与时间、风扇控制、网络、电源恢复策略、会话、用户管理、策略、证书。
- **已移除**（无后台支持）：转储页的“System dump”选项（只保留 BMC dump）、固件页 BMC 和 BIOS 两处的“备份镜像”卡片以及“切换为运行”（BMC 和 BIOS 都只有一个镜像区）、概览页“电源信息”卡片（功耗读数和功率上限依赖 DCMI 电源支持，本板不提供）、SNMP Alerts、清除密钥、LDAP、策略页的“虚拟 TPM”和“RTAD”开关、资源管理/电源、“仅重置服务器选项”、清单页的 DIMM/风扇/电源/处理器/组件表。

- **页面行为补丁** (`recipes-phosphor/webui/files/` , 编号 0016-0030) : POST Code 和事件日志默认最新在前; 传感器页分“模拟/离散”两个标签, 模拟传感器表分页; 固件页更新进度条在离开页面再回来后仍保留, BMC 更新分区需显式确认 (U-Boot / 环境变量分区默认不选), 并通过启动 ID 识别 BMC 已重启, 更新完成的通知保留到刷新或重新登录; 恢复出厂页只用 BMC 相关措辞; 概览页的固件卡只显示运行版本; 等待电源操作和实时状态页 (传感器等) 会自动刷新; 虚拟媒体停止时关闭会话, 大块读回复拆成 64 KiB 的 WebSocket 消息。
- **转储**: 只有 BMC dump (`phosphor-debug-collector`); 转储页走 bmcweb 的 Redfish Dump 服务, 需要编译选项 `redfish-dump-log` (已在 `bmcweb_%.bbappend` 里启用, 缺了这个选项转储页没有后端)。在 QEMU 里点“开始转储”要约 30 秒才完成, 期间再点会报“Another user initiated dump in progress”, 点一次后等它完成即可 (QEMU 里已验证列表正常)。
- **固件版本**: bmcweb 默认 (`redfish-updateservice-use-dbus=enabled`) 到 `/xyz/openbmc_project/software/bmc/functional` 找 BMC 版本, 而这里用的经典 `phosphor-image-updater` 发布在 `/xyz/openbmc_project/software/functional`, 结果 Redfish 的 `FirmwareVersion` 为空、网页 BMC 卡片显示 -- ; `bmcweb_%.bbappend` 里已把该选项设为 `disabled` (同时固件上传走 `/tmp/images`, 和经典更新服务一致) ⚠ 没有验证。 `journalctl` 里的 `mapperx: Found invalid association` 是 BMC 版本对象的 `inventory` 关联目标路径为空 (找不到 BMC 清单对象), 只是告警。网页只提供“从浏览器读取镜像文件” (走 bmcweb 的 `/vm/0/0` WebSocket → `jsnbd` → `nbd` → USB mass storage → 主机 VL805 USB 口); “从外部服务器读取镜像文件” (CIFS/HTTPS) 需要已停止维护的 `virtual-media` 服务, 镜像里没有, 网页默认也不显示。上板验证: 网页选一个 ISO 点开始, 主机里应出现一个 USB 光盘/U 盘; BMC 上 `ls /sys/kernel/config/usb_gadget/`、`ls /dev/nbd0`。
- **U-Boot 启动方式**: 固定从本地 SPI 闪存启动 (`bootcmd = run bootspi`), 不自动走网络; U-Boot 默认带静态网络参数 (192.168.185.200/24, 网关 192.168.185.1, TFTP 服务器 192.168.185.84; Linux 的 `eth0` 已改为 DHCP, 两者互不影响), 只用于手动 TFTP 启动调试和 `run netupdate`。虚拟机与物理主板的区别、环境重置、排查步骤详见第四章“4. U-Boot 的 TFTP 使用”。
- **时间和 SEL**: BMC 系统时间默认从板上 RTC (NCT3015Y) 读取, SEL 时间戳用系统时间。AST2600 内部 RTC 已关闭, NCT3015Y 是 `rtc0`。
- **SEL 记录**: 电压、温度 (含 `CPU_MAX_TEMP` / `DIMM_MAX_TEMP`, 含不可恢复级别)、watchdog 超时、BIOS 启动失败 (600 秒)、电源按键都会写 SEL。SEL 为 rollover, 用标准的 `logrotate` 实现 (`ceb-gnrd-sel-logrotate` , 开机 30 秒后首次检查, 之后每 1 分钟一次, 单个文件 15 KiB、保留 1 个旧文件), 约保留最新 100 到 200 条, 更老的删除; 按大小轮转, 条数是近似值, 记录 ID 由 `sel-log` 单独保存, 不会重复。
- **SSH / SCP**: BMC 用 dropbear 提供 SSH (22 端口), 已带 `openssh-sftp-server` 和 `openssh-scp`, `scp` 新旧协议都可用, 例如 `scp -P 2222 file root@127.0.0.1:/tmp/` (QEMU)。
- **SOL**: 走 AST2600 的 VUART1 (主机看到的是 COM1, I/O 0x3F8, 经 eSPI), 可双向; obmc-console 用 `ttyVUART0`, BIOS 的串口重定向要选 COM1。UART3 RX 仍保留但不再是 SOL 来源。网页 SOL 可以输入 (原来的只读补丁 `0005` 已移除)。
- **风扇控制**: 6 个风扇可单独或统一设置 (网页下拉框是“全部风扇”和 `SYS_FAN0` 到 `SYS_FAN5`), 模式只有“自适应” (最低 30%、最高 100%, 固定默认值, 没有最低转速滑块) 和“固定转速” (20/40/60/80/100%)。页面下方有命令框: 上面一个是读取每个风扇转速和模式的命令, 下面一个随当前选择实时生成设置命令。风扇控制器 (Pid) 在 Entity-Manager 里叫 `Fan0 Control` 到 `Fan5 Control`, 不能和风扇本身的 `SYS_FAN0` 到 `SYS_FAN5` 同名。

- **IPMI OEM 命令** (netfn 0x30, 只有两条, ipmitool 不用改, KCS 和 LAN 都可用, 已加入白名单; 由 `ceb-gnrd-ipmi-fan` 库实现, 转发给 `ceb-gnrd-fan-settings` 服务): `ipmitool raw 0x30 0x01` 读取, 返回 25 字节: 第 0 字节 “重启后保留” 标志, 之后每个风扇 4 字节 (模式 0 自适应/1 固定、占空比 % (0xFF 表示读不到)、RPM 低字节、RPM 高字节); `ipmitool raw 0x30 0x02 <风扇 0-5 或 0xFF 全部> <模式> <占空比十六进制> <保留 0/1>` 设置, 例如 `ipmitool raw 0x30 0x02 0xFF 0x01 0x3C 0x01` 是全部风扇固定 60% 并保留; 需要 Admin 权限。
- **网页保存的实现**: bmcweb 的 D-Bus REST 在这个版本里不能给方法传参数, Entity-Manager 对 Pid 属性的写入又会 “值已改但返回 InvalidArgs”, 所以网页每次保存只调用 `ceb-gnrd-fan-settings` 里一个不带参数的方法 `ApplyFan<0..5|All><Adaptive|Fixed20/40/60/80/100> <Keep|Forget>`, 方法名已包含风扇、模式和是否保留, 避免多客户端交错覆盖; 服务写 Entity-Manager 后会回读确认。勾选 “BMC 重启后保留这些设置” 时, 设置保存到 `/var/lib/ceb-gnrd/fan-settings.json`, 重启和断电重启后恢复, 不勾选则 BMC 重启后回到自适应。该功能依赖 bmcweb 的 `dbus-rest`。
- **升级后保留**: 普通固件升级不会清读写分区; 需要清读写分区的升级会按白名单保存时区、主机名、SSH 主机密钥、网站证书和风扇设置。恢复出厂则全部清除 (MAC 不受影响)。

4. IPMI

- `mc info`: Device ID 32, Device Revision 2, Product ID 3346 (0x0D12), Manufacturer ID 6659 (0x1A03), 在 BMC 上的 ipmitool 显示 `CTOPAI / CEB-GNR-D`。这些值来自 `recipes-phosphor/ipmi/phosphor-ipmi-config/dev_id.json`: `id` 是厂商自定义的设备编号 (沿用默认值 32, 不影响功能); `revision` 写 130 (0x82), 最高位 bit7 = 1 表示 “提供设备 SDR” (Provides Device SDRs: yes), 低 4 位 = 2 就是显示的 Device Revision。
- 传感器: 已启用 `dynamic-sensors`, 电压/温度/风扇/CPU_MAX_TEMP/DIMM_MAX_TEMP 都会出现在 IPMI。CPU_MAX_TEMP 告警阈值 90/98/105 °C, DIMM_MAX_TEMP 80/85/95 °C (UNC/UC/UNR, 只设上限); 6 个风扇不设告警, 没接风扇读 0 RPM 属正常; 温度读不到 (主机已开机) 时风扇 60% (temp-max 发布 70 °C, 两条曲线在 70 °C 都是 60%), 风扇读到几个都不影响 (FailSafePercent=30)。
- 主机侧 IPMI 走 KCS3 (见上); LAN 通道 1 是 eth0 (RMCP+ 只绑 eth0), 通道 2 是 NC-SI 的 eth1。
- SEL: 存放在 `/var/log/ipmi_sel`, `ceb-gnrd-sel-logrotate` (logrotate 按大小轮转) 保持 rollover, sel-logger 加了补丁, 把 “不可恢复” 事件也记成 SEL (CPU/DIMM 最高温的 UNR 放在我们自己的接口上, 不用 HardShutdown, 所以不会触发任何自动关机); 告警灯的四种种告警都有 SEL 记录。
- DCMI (`ipmitool dcmi power reading`、`dcmi get_temp_reading`): **未配置**, `power_reading.json` 的路径为空, `dcmi_sensors.json` 为空数组, 命令会报不支持或没有内容。
- 电压阈值: 规格表里的上下限 (标称 ±15%) 按 Critical 级别配置 (`lower critical / upper critical`), 所以 `ipmitool sensor` 的 LC / UC 列能显示; 越限时 sel-logger 直接记成 Critical 事件, 告警灯随之点亮。CPU/DIMM 最高温度的 UNC / UC / UNR 都能显示: UNR 在我们自己的接口 `com.ctopai.CebGnrd.Threshold.NonRecoverable` 上, ipmid 加了补丁读取它。**BMC 不会因为任何阈值自动关机**: 不使用 HardShutdown / SoftShutdown 接口, 并且镜像里去掉了会据此关机的 `phosphor-fan` 的 `sensor-monitor`。
- FRU 生成: `ipmitool fru gen [文件名]` (默认 `fru.bin`)。会依次提示 Chassis、Board、Product 三个区域的每个字段, 每项都显示含义/格式, 默认值已经填在输入行里, 可以直接修改, 直接回车则采用默认值。默认值: 制造商 `CTOPAI`, Board/Product 名称 `CEB-GNR-D`, Chassis PN `93-XXXX-XX`、

Board PN 91-59380-A0、Product PN 81-59380-A0，Product 版本 v1.0，Product 资产标签 Xeon6-S0C，Board / Product FRU file ID 0，序列号为生成当天 UTC 日期 YYYYMMDD 加 0001，例如 2610060001（序号可手动修改，不自动递增）。输入不合法会提示重输，标准输入不是终端时全部用默认值；字段是可打印 ASCII，最长 63 个字符，日期格式 YYYY-MM-DD 或 YYYY-MM-DD HH:MM（UTC，留空表示未指定），机箱类型填数字（默认 0x17 机架式）。生成后用 `ipmitool fru write 0 fru.bin` 写入主板 FRU（EEPROM 1 KiB，生成镜像须不超过 EEPROM 容量），再用 `ipmitool fru print 0` 核对。

- ipmid 启动：phosphor-ipmi-host 有一个 drop-in（10-ceb-gnrd-wait-sensors.conf），启动前最多等 90 秒，等映射器里的传感器数量连续 8 秒不变。原因是 ipmid 在传感器刚注册、阈值接口还没出来时去读会失败，开机后一分钟内 `ipmitool sensor` 只剩 2 个静态传感器。代价是开机后约一分钟内 `ipmitool` 不可用（QEMU 里已验证开机后传感器完整）。PCIe 槽位总线 i2c-0 至 i2c-5（本板没有 slot 2 的总线）。

七、验证清单

0. 一键检查脚本 ceb-gnrd-check

镜像里自带 `/usr/bin/ceb-gnrd-check`（源文件 `recipes-phosphor/utils/files/ceb-gnrd-check.sh`），在 BMC 里直接运行，逐项输出 PASS / FAIL，覆盖系统服务、传感器和阈值、IPMI 常规命令、风扇 OEM 命令（设置、读回、保留、清除、非法参数）、Redfish 和网页、RTC、MTD 分区等，同时把 journal、dmesg、D-Bus 树等日志打包。参数 0 表示检查 `run-qemu.sh` 的模拟器（默认），1 表示物理板；默认只读，结果分为 PASS / FAIL / SKIP / INFO（SKIP 表示需要额外操作，不能当成通过），有 FAIL 时退出码为 1。检查按已安装的板级配置核对各路 ADC、温度、风扇和已绑定的 PSU，主机关机时跳过只在开机时有效的读数。只有显式设置 `CEB_CHECK_CLEAR_LOGS=1` 才会清除事件日志。

```
ceb-gnrd-check 0 # 在 BMC 里运行；开机后等约 1 分钟再跑（ipmid 要等传
感器稳定）
BMC_PASSWORD='你的密码' ceb-gnrd-check 0 # root 密码不是默认值时
scp -P 2222 root@127.0.0.1:/tmp/ceb-gnrd-check.tar.gz . # 在 Ubuntu 上取回日志包（QEMU）
```

报告在 `/tmp/ceb-gnrd-check/report.txt`，诊断包在 `/tmp/ceb-gnrd-check.tar.gz`；重复运行会覆盖上一份，并发运行由 `/run/ceb-gnrd-check.lock` 阻止。该脚本在新固件上还没有重新跑过，也没有在物理板上运行过；接口元数据正常不等于数据路径全部通过。

1. 构建后先在 QEMU 里检查

```
systemctl --failed --no-pager # QEMU 缺少 KCS、eSPI、PECI 等硬件，对应服务失败属预期
journalctl -b -p err --no-pager | tail -40
ip addr show eth0 # QEMU 里 DHCP 第一个租约应为 192.168.185.200
cat /etc/os-release | head # VERSION_ID 应为 2.0.0；ipmitool mc info 的 Firmwar
e Revision 应为 2.00
ipmitool mc info # Manufacturer Name CTOPAI，Product Name CEB-GNR-D
```

网页逐项点开：菜单里不应再有 SNMP / 清除密钥 / LDAP / 资源管理；风扇页应列出 6 个风扇；SOL 页应能输入；KVM 页应有“全屏”按钮。

2. 上板后重点验证（对应端口指南红字项）

项目	命令 / 方法
eS PI	<code>dmesg grep -i espi</code> ; <code>cat /sys/kernel/debug/*espi*/regs</code> ; 对照分析仪抓包
PH Y	U-Boot: <code>mdio list</code> ; Linux: <code>dmesg grep -i -E "phy mdio"</code> 、 <code>ethtool -S eth0</code> 、 <code>iperf3</code>
PS U	插 1 个和 2 个模块各验证: <code>journalctl -u ceb-gnrd-psu-detect</code> ; <code>ipmitool sdr</code> 里应有 PSUn_Temp, <code>ls /sys/class/hwmon/*/temp*_input</code> 核对 temp2 确实是电源温度
电 源 按 键	<code>journalctl -u ceb-gnrd-power-button-log -f</code> ; <code>ipmitool sel list tail -3</code>
告 警 灯	电压越限、温度到 UNR、CPU_MAX_TEMP / DIMM_MAX_TEMP 到 Upper Critical、watchdog 超时、BIOS 启动超过 600 秒各验证一次（共用一个灯，阈值告警恢复后清除，BIOS 超时在后续 POST complete 成功后清除，watchdog 超时重启 BMC 才清除；没有其他故障时熄灭）； <code>busctl get-property xyz.openbmc_project.LED.GroupManager /xyz/openbmc_project/led/groups/enclosure_fault xyz.openbmc_project.Led.Group Asserted</code> 应随告警翻转， <code>cat /sys/class/leds/fault/brightness</code> 对应亮灭
风 扇	<code>busctl tree xyz.openbmc_project.EntityManager grep -i pid</code> ; <code>ls /xyz/openbmc_project/control/fanpwm/</code> ; 网页保存后查看 <code>journalctl -u ceb-gnrd-fan-settings</code> ; OEM 命令: <code>ipmitool raw 0x30 0x01</code> 、 <code>ipmitool raw 0x30 0x02 0xFF 0x01 0x3C 0x01</code> （再读一次确认），失败时 <code>journalctl -u phosphor-ipmi-host</code>
KC S	BMC: <code>ls /dev/ipmi-kcs3</code> 、 <code>journalctl -u phosphor-ipmi-kcs@ipmi-kcs3</code> ; 主机侧: <code>ipmitool -I open mc info</code>
NC -SI	主机上电后 <code>ip -br addr show eth1</code> 、 <code>journalctl -t ceb-gnrd-ncsi</code> （看有没有重试）；主机关机后接口应被关闭
SEL / 时 间	制造电压和温度越限， <code>ipmitool sel list</code> 里应有事件和解除事件；SEL 超过 2100 条后确认最老的被删；断电重启后 <code>date</code> 、 <code>ipmitool sel time get</code> 应保持 RTC 的时间
U- Bo ot	第一次要 <code>env default -a</code> ; <code>setenv ethaddr ...</code> ; <code>saveenv</code> , 确认 <code>printenv bootcmd</code> 是 <code>run bootspi</code> ; <code>run netupdate</code> 后 <code>reset</code>
BM C 状 态	<code>obmcutil state</code> （Device Available 取决于 BMC 是否为 Ready）

八、构建加速与常见问题

1. 缩短编译时间

- 不要随意 `cleansstate` , `bitbake` 会按内容判断哪些包要重编。
- 只改了一个包就单独构建, 补丁问题可以只跑 `bitbake -c patch <包>` (几秒) 。
- 把下载目录和共享缓存放在构建目录之外, 删掉 `build/` 重来也不必重新下载、重新编译, 在 `build/ceb-gnrd/conf/local.conf` 末尾加:

```
DL_DIR = "/home/test/yocto-cache/downloads"
SSTATE_DIR = "/home/test/yocto-cache/sstate"
```

2. 本项目遇到过的典型错误

现象	原因与处理
<code>patch-fuzz</code> QA 错误	补丁上下文与源码不完全一致; <code>webui-vue</code> 补丁必须前后各 3 行完整上下文, 用工具按实际文件生成, 不要手写 hunk 头
<code>malformed patch at line N</code>	hunk 头里的行数写错
<code>Missing Upstream-Status</code>	补丁说明里要有 <code>Upstream-Status:</code> 行
<code>do_patch</code> 里的 shell 追加报 <code>SyntaxError</code>	<code>do_patch</code> 是 Python 任务, shell 逻辑要写成独立任务并用 <code>addtask</code>
镜像大小超限	<code>FLASH_RWFS_OFFSET</code> 必须写成带 <code>override</code> 的 <code>FLASH_RWFS_OFFSET:flash-65536</code> , 普通赋值会被盖掉
打包阶段文件冲突	两个包安装了同一个文件 (如 <code>ipmitool</code> 自带的 IANA 企业编号表) , 改为在原包安装后追加
U-Boot 找不到 <code>.dtb</code>	2019.04 需要把 <code>ast2600-ceb-gnrd.dtb</code> 登记进 <code>arch/arm/dts/Makefile</code> (已放进 <code>0001-ceb-gnrd-board-device-tree-network-and-environment.patch</code>)
网页编译 <code>Unexpected token</code>	模板字符串反引号丢失, 补丁里的 JS 要逐字核对
<code>entity-manager: Probe statement failed to parse: True</code> , 电压、风扇、温度等传感器一个都没有	Entity-Manager 配置里的 <code>"Probe"</code> 只认大写的 <code>"TRUE"</code> / <code>"FALSE"</code> , 写成 <code>"True"</code> 整份配置会被拒绝; 用 <code>journalctl -b -p err grep -i entity</code> 看
<code>aspeed-g6-pwm-tach: Failed to create fan -22, fansensor: failed to find match for .../fanN_input</code>	6.18 的 PWM/TACH 驱动要求风扇子节点写 <code>tach-ch</code> 和 <code>pwms = <pwm_tach N 40000 0></code> , 旧写法 (<code>reg</code> 、 <code>aspeed,fan-tach-ch</code>) 会让它失败

现象	原因与处理
xyz.openbmc_project.LED.GroupManager.service: start operation timed out, identify 灯不工作	这个版本的 LED 管理器只读 /usr/share/phosphor-led-manager/led-group-config.json (或 /etc/phosphor-led-manager/ 下的同名文件), 找不到就一直等 Entity-Manager; 用 phosphor-led-manager_%.bbappend 把板级 led.json 装成这个文件名
ipmitool mc info 固件版本 0.00、网页运行版本 -, xyz.openbmc_project.Software.BMC.Updater 不存在	BMC 镜像更新守护进程没有自启 (updater 包整体设了 SYSTEMD_AUTO_ENABLE = "disable"), 它同时提供 BMC 版本对象和固件上传后端; 在 bbappend 里为它单独建 multi-user.target.wants 软链接
ModuleNotFoundError: No module named 'xml'	python3-dbus-fast 配方没带 python3-xml 等运行依赖, 用到它的 Python 服务要在自己的 RDEPENDS 里补
pin L26 already requested by ...mdio	EVb 设备树默认启用 mdio0 至 mdio3, 只保留用到的 mdio1, 其余禁用, 否则占用 I2C12 的引脚
风扇设置: busctl set-property ... Pid OutLimitMin 报 Invalid argument, 网页保存失败, ipmitool raw 0x30 0x02 返回 0xCC	Entity-Manager 写 Pid 属性时值已经改了却返回 InvalidArgs (它自己的日志里没有报错, 根因没有深究); Pid 与 AspeedFan 同名会共用一个 D-Bus 路径, 所以把 Pid 改名为 Fan<n> Control; ceb-gnrd-fan-settings 写完后回读确认, 值对了就算成功; 网页和 OEM 命令在 QEMU 里都已验证可用
ipmitool raw 0x30 0x01 返回 0xCE, 服务日志有 can't concat list to bytearray	dbus-fast 的 ay 返回值必须是 bytes, 不是整数列表
网页调 D-Bus REST 方法报 Invalid method arg type、对属性 PUT 报 Invalid arg type	这个 bmcweb 版本的 D-Bus REST 不能传标量参数; 网页只调用不带参数的方法 (见第六章 “风扇控制”)
开机后约一分钟内 ipmitool sensor 只有 2 个传感器, 日志有 Failed to update sensor map for threshold sensor	ipmid 在传感器的阈值接口出现前去读; phosphor-ipmi-host 的 drop-in 让它等传感器数量稳定后再启动
Redfish FirmwareVersion 为空、网页 BMC 版本 --, 但 busctl 里版本对象存在	bmcweb 默认从 /xyz/openbmc_project/software/bmc/functional 取版本, 经典更新服务发布在 /xyz/openbmc_project/software/functional; bmcweb_%.bbappend 里设 -Dredfish-updateservice-use-dbus=disabled (strings /usr/bin/bmcweb grep -c software/bmc/functional 非 0 说明用的是新路径)
swampd: Must have one stepwise point, 风扇不按温度调速	Entity-Manager 把整数数组 ([40, 50, ...]) 以 at 发布, pid-control 读不到, Stepwise 的 Reading / Output 要写成带小数点的 40.0

现象	原因与处理
启动日志里有 <code>mmc0: Failed to initialize a non-removable card</code>	EVB 设备树启用了 eMMC/SD; 本板没有, U-Boot 和 Linux 设备树里都把 <code>emmc</code> 、 <code>sdhci</code> 相关节点禁用
<code>patch-fuzz</code> 警告出现在 <code>phosphor-ipmi-host</code>	手写补丁的上下文不对; 按真实源码重新生成补丁 (<code>0001</code> 已重新生成)
<code>os-release.bb: git describe --dirty ... No names found</code>	openbmc 仓库没有带注释的 tag; <code>git tag -a v2.0.0 -m "CEB-GNRD 2.0.0"</code> (必须用 <code>-a</code>), 固件版本号仍由 <code>DISTRO_VERSION</code> 决定 (定义在 <code>meta-ctopai/conf/distro/ctopai-openbmc.conf</code> , <code>local.conf</code> 里必须是 <code>DISTRO ?="ctopai-openbmc"</code> , <code>./setup ceb-gnr</code> 会自动把旧的 <code>openbmc-phosphor</code> 改过来; 没改的话固件版本会变成 <code>git describe</code> 的结果)
告警灯服务日志反复出现 <code>Unable to read host boot GPIOs: Command '['gpioget', 'gpiochip0', '172']' returned non-zero exit status 1</code>	第 172 号线是 <code>BMC_CPU_PWRGD</code> , 已被 <code>x86-power-control</code> 独占请求, <code>gpioget</code> 会报设备忙。告警灯服务改为读 D-Bus 的 <code>xyz.openbmc_project.State.Chassis.CurrentPowerState</code> (<code>x86-power-control</code> 根据这根线发布的状态); <code>BMC_BIOS_BOOT_OK</code> 现在也是 <code>x86-power-control</code> 的 <code>PostComplete</code> , 同样改读 <code>OperatingSystemState</code> , 告警灯服务不再直接读任何 GPIO
<code>wrong LCONF_VERSION (has 8, expecting 7)</code>	发行版文件 <code>ctopai-openbmc.conf</code> 在 <code>meta-ctopai</code> 层里, 而旧的 <code>build/ceb-gnr/conf/bblayers.conf</code> 只列了 <code>meta-ceb-gnr</code> , 没有 <code>meta-ctopai</code> ; <code>DISTRO</code> 找不到发行版文件时 BitBake 不报错 (可选包含), <code>LAYER_CONF_VERSION</code> 就退回 <code>oe-core</code> 默认的 7 (<code>phosphor-base.inc</code> 里设的是 8)。把 <code>meta-ctopai</code> 加进 <code>BBLAYERS</code> (<code>./setup ceb-gnr</code> 会自动补), 或在 <code>bblayers.conf</code> 里手动加一行 <code>/home/test/openbmc/meta-ctopai \</code>
<code>ipmitool sel list</code> 一直是 "SEL has no entries", 但 <code>journal</code> 里有 <code>sel-logger ... threshold assert</code>	<code>rsyslog</code> 没有读 <code>journal</code> : 默认只加载 <code>imuxsock</code> , 拿不到 <code>MESSAGE_ID</code> 、 <code>IPMI_SEL_*</code> 字段, <code>/var/log/ipmi_sel</code> 就从来没被写入。 <code>rsyslog.conf</code> 里要 <code>PACKAGECONFIG:append = " imjournal"</code> (Intel 参考平台同样这么做); 开机日志里应有 <code>imjournal</code> , 没有 <code>Acquired UNIX socket ... syslog</code> 之外的输入就说明没生效
<code>ipmitool sel list</code> 有记录, 但网页 "Event logs" 是空的	网页读的是 Redfish 事件日志 <code>/var/log/redfish</code> (格式 <code><时间> <MessageId>,<参数></code>), 和 IPMI 的 SEL (<code>/var/log/ipmi_sel</code>) 是两个文件。需要 <code>rsyslog</code> 有一条规则, 把带 <code>REDFISH_MESSAGE_ID</code> 的 <code>journal</code> 条目写进去 (<code>rsyslog.conf</code> 里要 <code>PACKAGECONFIG:append = " imjournal"</code> 安装的 <code>ceb-gnr-redfish.conf</code> , Intel 参考平台同样的规则, 同样依赖 <code>imjournal</code>)。只有 <code>bmcweb</code> 注册表里有的消息 ID 才会显示
<code>do_rootfs: Group render has never been defined</code>	来自 <code>rootfs-postcommands.bbclass</code> 的 <code>systemd_sysusers_check</code> , 核对 <code>/usr/lib/sysusers.d/*.conf</code> 声明的组是否都在 <code>/etc/group</code> 。 <code>render</code> 组本来由 <code>udev</code> 包创建, 但 <code>meta-phosphor</code> 的 <code>systemd.conf</code> 有意把 <code>udev</code> 从 <code>USERADD_PACKAGES</code> 里去掉 (为了让 <code>udev</code> 不依赖 <code>shadow</code> , 能放进 <code>initramfs</code> , BMC 没有 <code>/dev/dri</code>), 所以这个告警是预期的, 无影响, 忽略
<code>systemd: Failed to set watchdog pretimeout_governor to 'panic'</code>	<code>aspeed_wdt</code> 不支持预超时, 内核不提供 <code>pretimeout_governor</code> (打开 <code>CONFIG_WATCHDOG_PRETIMEOUT_GOV_PANIC</code> 没有用); <code>systemd.conf</code> 里同名的 <code>40-hardware-watchdog.conf</code> 覆盖 <code>meta-phosphor</code> 的那个, 去掉 <code>RuntimeWatchdogPreSec</code> / <code>RuntimeWatchdogPreGovernor</code>

3. 修改网页补丁的建议流程

1. 从 GitHub 下载对应版本的网页源码文件，生成改动后的文件。
2. 用工具生成带 3 行上下文的标准补丁（不要手写 hunk 头）。
3. 在构建机上先 `bitbake -c patch webui-vue`，通过后再完整构建。

九、板卡层实现说明（原 `meta-ceb-gnrd/README.md`）

本章是板卡层 `meta-ctopai/meta-ceb-gnrd` 的实现说明（原来的层 README，现已译为中文并按最新代码核对），和第六章互为补充：第六章按功能列出现状，本章说明各项的实现方式、涉及的文件和还需要上板确认的事项。逐个信号的状态见同目录的 `port_guide.xlsx`。除非注明，本章内容没有在物理板上验证过，仍需要板子的事项集中在本章末尾的“待确认事项”。

参考实现：**meta-ibm/meta-sbp1**（Intel 服务器平台）。构建、QEMU 运行、TFTP 加载和验证方法见第三章到第七章。

平台功能

Flash 布局与升级

- BMC Flash: W25Q512JVFIQ, 64 MiB, 接在 AST2600 的固件 SPI (FMC)。`FLASH_SIZE = 65536`，`FLASH_RWFS_OFFSET:flash-65536 = "55296"`。

u-boot	0x00000000	0xe0000	
u-boot-env	0x00e00000	0x20000	
kernel	0x01000000	9 MiB	(FIT: 内核、设备树、initramfs)
rofs	0x0a000000	44 MiB	(squashfs)
rwfs	0x36000000	10 MiB	(jffs2, 设置)

Linux 和 U-Boot 的设备树必须保持这些偏移；U-Boot 里的 `netupdate`（见下）使用同样的数字。

- 只有一个镜像区，Web 界面不显示备份镜像。
- 固件更新包（`*.static.mtd.tar`：image-u-boot、image-kernel、image-rofs、image-rwfs）先暂存到 `/run/initramfs`，BMC 重启时由 initramfs 的更新脚本写入：
 - kernel 和 rofs 被替换；
 - rwfs 被重写，只恢复 OpenBMC 白名单（用户和密码、IPMI 密码、网络、DNS、设置）和 `recipes-phosphor/initrdscripts/files/ceb-gnrd-whitelist`（时区、主机名、SSH 主机密钥、网站证书、风扇设置、含网页登录会话的 bmcweb 数据）里的文件，所以网页保持登录；SEL 和事件日志会丢失；

- 默认**不**重写 U-Boot (`ceb-gnrd-update-skip-u-boot.sh` ; 写 U-Boot 时断电会得到一块起不来的板子) ; 要有意更新它, 需要在重启前创建 `/run/initramfs/update-u-boot` ;
- U-Boot 环境 (MAC 地址) 不属于更新包。
- 在 Web 固件页选择 BMC 的 `.static.mtd.tar` 分区更新包后, 页面显示分区选择: `kernel/rofs` 默认选中且必须一起更新; `rwfs` 默认选中 (空镜像则跳过, 保留现有 `rwfs`) ; `u-boot` 和 `u-boot-env` 默认不选, 勾选时必须确认 (写 U-Boot 失败可能无法启动, 清除环境会丢失保存的 MAC) 。选择以 `bmc-partitions.txt` 随更新包传递, 固件激活时再次校验, 只暂存被选中的分区; 整片 `.static.mtd.all.tar` 更新被拒绝。
- 恢复出厂会清除 `rwfs` 里的全部内容 (MAC 不受影响) 。

U-Boot

- 启动顺序: `bootcmd` 固定为 `run bootspi` , 从本地 SPI Flash 里的 FIT 镜像启动, U-Boot 不会自己通过网络加载系统。TFTP 只在手动操作时使用: `tftpboot 0x83000000 fitImage` 加 `bootm` 用于调试内核 (根文件系统仍来自本地 `rofs`) , 以及下面的 `run netupdate` 。
- 默认网络参数是静态的: 192.168.185.200/24, 网关 192.168.185.1, 服务器 192.168.185.84 (Linux 的 `eth0` 已改为 DHCP, 二者不再一致; 在 `aspeed-common.h` 中由 `0001-ceb-gnrd-board-device-tree-network-and-environment.patch` 设置, 该补丁同时登记设备树并把板级环境加入默认环境) 。
- 默认 MAC: `ethaddr=02:26:00:00:00:01` 、 `eth1addr=02:26:00:00:00:02` (`recipes-bsp/u-boot/files/ceb-gnrd-env.h`) 。已保存的环境里缺这两个变量时, `board_late_init()` (`0003-ceb-gnrd-fill-missing-default-mac.patch`) 在网卡初始化前补齐并保存一次, 已有的非空值不变。真实板卡应写入每板唯一的 MAC。
- `run netupdate` (变量定义在 `ceb-gnrd-env.h`) 通过 TFTP 取 `image-kernel` 和 `image-rofs` , 检查大小后用 `sf update` 写入 `kernel` 和 `rofs` 分区, 不会写 U-Boot、环境和 `rwfs` 。
- 环境保存在 Flash 中, 之前保存的环境 (例如旧的先走 TFTP 的 `bootcmd`) 会盖住新的默认值: 第一次要执行 `env default -a; saveenv` , 并设置 `ethaddr` 。
- `0002-ceb-gnrd-pass-boot-reset-cause-to-linux.patch` 把启动时的原始复位原因 (上电 / 看门狗等) 通过设备树 `/chosen` 传给 Linux, 供 “只在 AC 上电时应用来电恢复策略” 使用; 只更新 `kernel/rofs` 而不更新 U-Boot, 复位原因会显示 `unknown`, 来电恢复策略会被跳过。
- U-Boot 只使用 RGMII 口。NC-SI 的 MAC (`&mac2`) 在 U-Boot 设备树里禁用: 加上 `phy-mode` 后探测会让 U-Boot 崩溃 (data abort, 不断复位) , 不加则只打印 “Invalid PHY interface” 。
- 在 QEMU 里 TFTP 服务器是 192.168.185.1 (QEMU 的 `tftp=` 选项) , 不是 192.168.185.84, `PHY/mii` 行为是模拟的 (RGMII 时序和 RTL8211FS 延时要在板上确认) 。
- BMC 的 DDR4 (SK hynix H5AN8G6NDJR-XNC, 1 GiB, 单颗 x16) 参数在 `recipes-bsp/u-boot/files/ceb-gnrd-ddr4.cfg` ; DDR 初始化变化必须更新 U-Boot/SPL, 仅更新 `kernel/rofs` 不会修改已有启动代码。

BIOS 交互 (BIOS 菜单)

- **PLDM over MCTP** (`conf/distro/include/pldm.inc`) : 通过 Redfish `Systems/system/Bios` 提供 BIOS 属性表和 `setup` (菜单) 设置, 另有 PLDM 传感器和 PLDM 固件更新。需要 BIOS 支持 PLDM。

- **biosconfig-manager**: 通过 BMC 远程查看和修改 BIOS setup 参数 (参见 <https://github.com/openbmc/bios-settings-mgr>) 。
- **phosphor-host-postd + phosphor-post-code-manager**: I/O 端口 0x80 的 BIOS POST 码流水线。AST2600 的 LPC snoop 节点对端口 0x80 使能; POST 捕获要在硬件上验证。LPC snoop 只是监视, 不是 eSPI Peripheral I/O 周期的完成路径。
- **phosphor-software-manager + flash_bios**: 通过接在 AST2600 SPI1 (单线模式) 上的 Macronix MX25U51245GMI00 (64 MiB) 更新主机 BIOS。主机开机时, 更新服务会在服务日志里提示, 并最多等 30 分钟让主机进入稳定的关机状态, 运行中绝不会抢占 BIOS Flash 的控制权。随后它把 GPIOM1 (`BMC_BIOS_FLASH_SELECT`) 拉高, 把 BIOS Flash 切给 BMC, 再查找 MTD (必要时重试 SPI-NOR 探测), 等 5 秒让 CPLD 把 CPU 置于 S5, 然后刷写整颗 `host-bios` MTD。结束后恢复 BIOS 的所有权并发出一次强制关机的电源按钮脉冲, 再请求开机。如果主机仍为开机状态, 常规的机箱关机请求会产生配置的 8 秒强制脉冲; 如果已经是关机/S5, `x86-power-control` 里受保护的板级方法发出该脉冲, 且不会从守护进程那里抢走 GPIO 所有权。要在装配好的板子上确认 GPIO 极性和上电时序。只写被选择的 Flash 区域 (`flashrom -l <layout> -i <region>` , 同时会跳过没变的块), 其他区域内容不变。板级布局固定为 `/usr/share/ceb-gnrd/bios-layout.txt` (descriptor、metadata、pdr、bios、nac1、nac0、reserved, 网页上有同一张表)。要写的区域来自更新包里的 `bios-regions.txt` , 由网页固件页的复选框添加; 没有它时 (curl、Redfish 客户端) 除 nac0/nac1 外的所有区域都会写。nac0/nac1 保存 CPU 集成网络控制器的设置和 MAC 地址, 网页只在确认对话框之后才允许选择。镜像必须是完整的 64 MiB Flash 镜像。每一步都会把百分比写到更新对象的 `ActivationProgress` , bmcweb 把它转成 Redfish 更新任务的 `PercentComplete` , 网页固件页 (补丁 0013) 显示带步骤名的进度条; 数字列在 `bios-update.sh` 里, 必须与该补丁里的表一致。
- **phosphor-ipmi-flash**: 通过 BLOB 协议的 IPMI 带内固件更新 (启用 `flash_bios` PACKAGECONFIG 时带 `host-bios` 目标) 。
- **主机/机箱状态管理**: `MACHINE_FEATURES` 包含 `obmc-host-state-mgmt` 、 `obmc-chassis-state-mgmt` 、 `obmc-phosphor-chassis-mgmt` 、 `obmc-phosphor-flash-mgmt` 。主机和机箱状态来自 `x86-power-control` (`power-config-host0.json` : `PowerOk` 为 `BMC_CPU_PWRGD` , `PowerOut` 为 `BMC_CPU_POWER_BUTTON` , `ResetOut` 为 `BMC_CPU_RESET` , 电源脉冲 200 ms、强制关机 8 s、复位 500 ms) 。机器使用 `obmc-bsp-common.inc` (受管服务器), 而不是 `obmc-evb-common.inc` 。
- 机箱电源按键输入 `BMC_POWER_BUTTON_INPUT` 只检测: 按下时写入 SEL 和日志 (`ceb-gnrd-power-button-log`) , 不触发电源状态切换, 也不直通到 CPU 电源按键输出。
- Redfish 侧需要 BIOS 提供的内容 (通过 IPMI 报告的启动进度、通过 Get System Boot Options 的一次性启动设备、SEL 写入、用于系统/CPU/内存清单的 SMBIOS 交接、`Bios` 用的 PLDM) 本层都不提供, 所以网页里去掉了系统、处理器和内存清单表。

eSPI 与主机 IPMI (KCS)

- 板子把 AST2600 的 eSPI 接到 Xeon 6 主机。GPIO W0–W7 专用于这条连接: `pinctrl_espi_default` 覆盖 W0–W5/W7, 另外的 `pinctrl_espialt_default` 覆盖 W6/AD7, 二者都由 eSPI 节点选用。Peripheral 和 Virtual Wire 通道已使能 (主机是 Intel PCH, 总是使用 Virtual Wire), Flash Access 不用。KCS/IPMI、POST 码和 VUART SOL 不需要 OOB (VUART 由 BMC 配置) 。
- 固定版本的 `linux-aspeed` (`c0538446`) 没有 AST2600 eSPI 控制器驱动。本板自带一个精简驱动 (`0001-soc-aspeed-add-AST2600-eSPI-peripheral-ready-driver.patch` , 其 Makefile 一行也在这个补丁里), 并使能 `CONFIG_ASPEED_ESPI` 。它使能 Peripheral 和 Virtual Wire 通道: 置位 Peripheral 与

Virtual Wire 的 Software Ready 以及 slave boot done / status 系统事件 (ESPI098 的 bit 20 和 bit 23, 主机等待它们), 在主机 eSPI 复位时复位该模块并把上述全部重新设置 (线路本身、GPIO 和系统事件保持硬件模式, 没有软件模式, 也没有 Virtual Wire 中断), 统计通道错误和中止, 记录状态变化 (限速), 并提供 debugfs 的 regs 转储。主机连不上 BMC 时先看 `dmesg | grep -i espi`。该内核没有 `CONFIG_ASPEED_LPC_SIO`, 也没有添加。

- Peripheral I/O 周期在 Peripheral Ready 置位后由 AST2600 硬件完成; `ASPEED_LPC_SNOOP` 只是端口 0x80 的监视器。
- 主机 IPMI: ASPEED KCS BMC、IPMI 和 raw cdev 选项, `phosphor-ipmi-kcs` 使用默认的 `ipmi-kcs3` 设备。设备树启用 `&kcs3` 并带 `aspeed,lpc-io-reg = <0xca2>`, 没有这个属性内核驱动不会探测。KCS 没有配置 SerIRQ, 主机轮询状态寄存器。BIOS 必须把它的 BMC KCS 端口设为 0xCA2。
- GPIO7 (`BMC_HBLEDD#`) 初始为灭。eSPI Peripheral 驱动绑定并置 `SW_READY` 之后, 由一个板级服务启用内核 LED 的 heartbeat 触发器。

PECI、温度与风扇控制

- AST2600 PECI0 用于 BMC_CPU_PECI 连接 (封装球 AT29)。内核使能 `CONFIG_PECI`、`CONFIG_PECI_CPU`、`CONFIG_PECI_ASPEED`、`CONFIG_SENSORS_PECI_CPUTEMP`、`CONFIG_SENSORS_PECI_DIMMTEMP`。官方 `peci-cpu` 驱动不认识 Granite Rapids (Xeon 6, CPUID model 0xAD / 0xAE), 内核补丁 `0003-peci-add-Granite-Rapids-CPU-and-DIMM-temperature.patch` 为 GNR/GNR-D 增加了 CPU 封装温度 (跳过核心掩码) 和 DIMM 温度配置: GNR-D 的 PCS 14 提供八个通道的最高温度, 不再使用 SPR/EMR 的 DIMM 阈值寄存器。如果 BIOS 使用 CLTT with PECI wire / PECI_UPDATE, PCS 14 返回零, 该模式的 DIMMTEMPSTAT MMIO 路径还缺寄存器规范, 不能声称已覆盖。读到全零或读取出错时内核温度保持不可用。以上都没有在板上验证: 检查 `ls /sys/bus/peci/devices/0-30` 是否有 `peci_cputemp.*` / `peci_dimmtemp.*` 及其读数。IntelCPUSensor 不编译 (`PACKAGECONFIG:remove`, `ceb-gnrd.json` 里没有 XeonCPU 条目): IPMI、Redfish 和网页里不会出现逐核或逐 DIMM 的传感器, 只有下面的两个最大值。只启用了 AST2600 的 I3C3, DIMM 温度走 PECI, 不走 I3C。
- `ceb-gnrd-temp-max` (Python, dbus-fast) 发布两个传感器 `/xyz/openbmc_project/sensors/temperature/CPU_MAX_TEMP` 和 `DIMM_MAX_TEMP`, 取内核 `peci_cputemp` (CPU) 和 `peci_dimmtemp` (DIMM) hwmon 设备温度的最大值, 直接读 sysfs (按标签排除 DTS、Tcontrol、Tthrottle、Tjmax 和 margin 读数)。主机关机时为 0; 主机开机但读不到任何温度时为 70 °C (两条风扇曲线在 70 °C 都映射到 60 %, 不告警)。只设上限 (非严重 / 严重 / 不可恢复): CPU 90 / 98 / 105 °C, DIMM 80 / 85 / 95 °C。前两级使用 Warning 和 Critical 阈值接口; 不可恢复级在一个私有接口 (`com.ctopai.CebGnrd.Threshold.NonRecoverable`) 上, 绝不用 HardShutdown: BMC 不得因为阈值关闭系统, 而 `phosphor-fan` 的 sensor monitor 等服务会在 HardShutdown 告警时给系统断电 (该 monitor 也已从镜像里去掉)。服务发出 `ThresholdAsserted` 信号, 由 `phosphor-sel-logger` 转成 SEL 记录; `ipmid` 的板级补丁把不可恢复值显示为 UNR。发现的源传感器名称会记录日志 (`journalctl -u ceb-gnrd-temp-max`), 需要在板上核对。
- 风扇由 `phosphor-pid-control` 按 Entity-Manager 配置 (`ceb-gnrd.json`) 驱动: 6 个风扇 PID 控制器 (`Fan0 Control` 到 `Fan5 Control` , 输入 `SYS_FAN0-5`, 输出 `PWM0-PWM5`, 限幅 30-100 %), 一个区域 (`MinThermalOutput 30`), 以及作用在 `CPU_MAX_TEMP` 和 `DIMM_MAX_TEMP` 上的两条 stepwise 曲线。曲线点是占位值, 等待确认。区域的 fail-safe 故意设为 30 %: 能读到的风扇数量不应决定风扇转速, 温度传感器才决定 (读不到 CPU 或 DIMM 温度: 60 %)。6 个风扇没有转速告警, 没接风扇的接口读数为 0 RPM。Pid 对象的名字不能与 AspeedFan 对象 (`SYS_FAN0-5`)

相同：二者会共用一个 D-Bus 路径，对 Pid 属性的写入会失败。Stepwise 的 Reading / Output 数组要写成带小数点（40.0），因为 pid-control 读不了 Entity-Manager 为 40 发布的无符号整数数组。

- `ceb-gnrd-fan-owner` 在 pid-control 运行且 6 路 PWM/TACH 都存在后，把风扇从 CPLD 交给 BMC（GPIO16 `BMC_FAN_BMC_OVERRIDE_N`），pid-control 停止时再交回。BMC 复位（看门狗或用户触发）期间必须把风扇交还 CPLD，所以这根脚不能保持：内核对用户态申请的线都会置位 `reset tolerance`，服务在占住这根线之后用 `devmem` 清除 `0x1e7800ac` 的 bit 6，清不掉就不接管风扇。
- `ceb-gnrd-fan-settings` 支撑网页的风扇页（逐个或统一的限幅，可选的跨 BMC 重启保留，存放在 `/var/lib/ceb-gnrd`），并每 30 秒把保存的限幅重新应用到 Entity-Manager 的 Pid 对象。自适应模式使用固定限幅（30 % 到 100 %），没有最低转速设置。Entity-Manager 对 Pid 属性的写入会返回 `InvalidArgs` 但值其实已改变（根因没有深究），所以服务回读该值，一致就接受。这个版本的 bmcweb D-Bus REST 不能传标量参数，所以网页只发一次不带参数的调用，方法名承载整个请求：`ApplyFan<0..5|All><Adaptive|Fixed20..100><Keep|Forget>`。
- 同样的控制也以两条 IPMI OEM 命令提供（`netfn 0x30`）：`0x01` 读取（标志字节，然后是 `SYS_FAN0.5` 各自的模式/占空比/RPM 低字节/RPM 高字节，共 25 字节，占空比 `0xFF` 表示未知）和 `0x02` 设置（风扇 0-5 或 `0xFF`、模式、占空比、是否保留；需要 Admin）。它们由 `ceb-gnrd-ipmi-fan` 这个 ipmid provider 实现（由 `ceb-gnrd-ipmi` 引入），并在 `ceb-gnrd-ipmi-whitelist.conf` 里放行。曾在 QEMU 里用 `ceb-gnrd-check` 检查（设置、读回、保留、清除、非法风扇号）；还没有在板上检查。

启动、ipmid 启动与板级自检

- AST2600 的 SD/eMMC 控制器在 U-Boot 和 Linux 设备树里都被禁用（EVB 的 include 会启用它们），本板两者都没有。
- 硬件看门狗：WDT1 只复位 SoC（`aspeed,reset-type = "soc"`，不是整颗芯片，所以 GPIO 保持状态，需上板确认），由 systemd 喂狗（`RuntimeWatchdogSec=120s`）。`aspeed_wdt` 不支持预超时，所以 `systemd-conf` 安装一个与 meta-phosphor 同名的 `40-hardware-watchdog.conf`（/etc 里的优先），只保留 `RuntimeWatchdogSec=120s`，去掉 `RuntimeWatchdogPreSec` / `RuntimeWatchdogPreGovernor=panic`（否则 systemd 每次开机都会提示“Failed to set watchdog pretimeout_governor”；用空赋值清除会被 systemd 拒绝）。
- 内核 oops 变成 panic（`CONFIG_PANIC_ON_OOPS`），panic 后 5 秒重启 BMC（`CONFIG_PANIC_TIMEOUT=5`）。启用了 Magic SysRq（串口 BREAK 不能触发），可用 `echo c > /proc/sysrq-trigger` 测试。
- 服务恢复使用标准的 systemd / OpenBMC 机制（`ceb-gnrd-health`）。本层自己的 fan-settings、temp-max 和 alert-led 服务只加一个 drop-in：`Restart=always` 和启动限制（5 分钟内 5 次），不加别的。systemd 在每次失败时都会执行 `OnFailure=`，包括随后会自动重启的失败（在虚拟机上见过：给 temp-max 发一次 SIGKILL 就让 BMC 进入 Quiesced 并重启），所以它们不能带 `OnFailure=`。对象映射器、Entity-Manager、bmcweb 和 ipmid 加同样的内容，再加 `OnFailure=obmc-bmc-service-quiesce@0.target`，所以它们之一第一次失败时（它也会被重启，但 BMC 无论如何都要重启），phosphor-state-manager 把 BMC 置为 Quiesced；选项 `auto-reboot-on-bmc-quiesce`（`phosphor-state-manager_%.bbappend`）再把它重启。上游对这种重启没有次数限制；`ceb-gnrd-quiesce-reboot-limit.sh`（`phosphor-bmc-quiesce-reboot.service` 的 ExecCondition）限制为最多自动重启 1 次，计数存在读写 Flash 里，开机 15 分钟后由 `ceb-gnrd-quiesce-reboot-clear.timer` 清零。重启后故障仍在，BMC 就停在 Quiesced（并写一条 SEL）等待人工处理。上游守

护进程不向 systemd 看门狗报活，所以对它们只能恢复崩溃（进程还在但卡死的情况发现不了）；本层自己的服务是 `Type=notify` 加 `WatchdogSec=`，由主循环报活，卡死会被重启。`ceb-gnrd-wdt-reset-log` 在 `bootstatus` 显示看门狗复位而没有干净关机标记时写一条 SEL（在 U-Boot 里输入 `reset` 也会被记一次）。以上新增部分没有编译，也没有运行验证。

- `phosphor-ipmi-host` 有一个 drop-in（`10-ceb-gnrd-wait-sensors.conf`），最多等 90 秒，直到 D-Bus 传感器数量连续 8 秒不变。更早启动时，`ipmid` 会在传感器的阈值接口出现前读取，开机后第一分钟只提供 2 个静态传感器。代价是开机后大约一分钟内 `ipmitool` 不可用。
- `ceb-gnrd-boot-progress`（`recipes-phosphor/state`）根据端口 0x80 的 POST 码发布主机启动进度（`/xyz/openbmc_project/state/host0` 上的 `xyz.openbmc_project.State.Boot.Progress`），因为上游没有任何组件推导它，主机也不上报。POST 码范围（脚本里的 `STAGES`）参照公开的 AMI Aptio 检查点和 Intel MRC，以 GNR-D BIOS 厂商的 POST 码表为准。IPMI 的 `Boot_Progress` 传感器和网页的离散传感器表会显示它；Redfish 的 `BootProgress` 由 `bmcweb` 从 `x86-power-control` 读取，仍为空。
- POST 码管理保留所有实际收到的码值（包括 `0x00`、重复的 `0x01` 和多字节码），只保留最近 2 轮 BIOS POST、每轮最多 512 条（`0001-ceb-gnrd-post-history-warm-boot-cycle.patch`，旧的 100 槽历史由 `ceb-gnrd-post-history-limit.py` 迁移）；收到主机 `CurrentHostState=Off` 时结束当前一轮，下一条记录开始新一轮；网页上最新在前。
- `ceb-gnrd-check`（`recipes-phosphor/utils`，安装到 `/usr/bin`）在 BMC 上运行，输出服务、传感器和阈值、IPMI 命令、风扇 OEM 命令、Redfish、RTC 和 MTD 布局的 PASS/FAIL，并把日志打包到 `/tmp/ceb-gnrd-check.tar.gz`（用法见第七章）。
- `bmc-hw-dump`（`recipes-phosphor/utils/files/bmc-hw-dump.sh`，安装到 `/usr/bin`）是只读转储，记录正在运行的固件如何使用硬件（GPIO、pin mux、I2C、eSPI/KCS/VUART、网络、Flash 等）。把脚本复制到旧的厂商固件上运行，再在本固件上运行 `bmc-hw-dump`，然后在 PC 上用 `sh bmc-hw-dump.sh compare OLD.tar.gz NEW.tar.gz` 对比。转储里的 `ceb-gnrd-checklist.txt` 逐项列出每个 `ceb-gnrd` 硬件功能的期望值和实际值。

与 OpenBMC 惯例保持一致

- x86 平台的设置参照 Intel 参考平台：`obmc-host-ctl` 不是机器特性（它唯一的提供者是 OpenPOWER 的 `obmc-op-control-host`），`VIRTUAL-RUNTIME_obmc-discover-system-state` 是 `x86-power-control`，BMC 启动时由它应用来电恢复策略，且只在 AC 上电启动时应用（`0003-ceb-gnrd-apply-power-restore-only-on-ac-boot.patch`）：BMC 单独复位时保持主机现有状态。策略的出厂默认值是 `AlwaysOn`（AC 恢复时主机开机），由 `phosphor-settings-manager/settings.override.yml` 设置；读写分区里已经保存的策略（在网页上改过的）会一直保留到恢复出厂。
- 私有的 D-Bus 名字使用厂商域名：服务和接口为 `com.ctopai.CebGnrd.*`（`FanSettings`、`TempMax`、`Threshold.NonRecoverable`）。风扇设置对象的路径仍是 `/xyz/openbmc_project/ceb_gnrd/fan_settings`，因为 `bmcweb` 的 D-Bus REST 只提供 `/xyz` 和 `/org` 下的对象路径。
- 对上游源码的改动都用补丁文件，不用 `sed`：U-Boot（`0001-...` 到 `0003-...`）、内核 Makefile 一行（在 eSPI 补丁里）、`ipmitool` 的产品名（`0002-ipmitool-add-ceb-gnrd-product-name.patch`）。`x86-power-control` 和 `ipmid` 的补丁已对照固定版本的源码重新生成，所以 `patch-fuzz` QA 降级已经消失。`ipmitool` 的厂商名（IANA 企业编号 6659）仍然来自在 `do_install:append` 里往已安装的 `enterprise-numbers` 数据文件追加的一行，因为该文件不是 `ipmitool` 源码的一部分。

- `DISTRO_VERSION` 定义在厂商发行版 `ctopai-openbmc` (`meta-ctopai/conf/distro`) 里; `local.conf` 必须选用它 (`./setup ceb-gnrd` 会把旧的 `DISTRO ?= "openbmc-phosphor"` 改过来)。板级硬件契约通过 `MACHINE_EXTRA_RDEPENDS` 安装, 而不是机器特性。
- 有意不改的部分: `bmcweb` 保持 `redfish-updateservice-use-dbus=disabled`, `phosphor-software-manager` 保持经典更新器 (去掉了 `software-update-dbus-interface`, BMC 更新器通过软链接启用)。默认流程用 `Software.Manager` 取代 `xyz.openbmc_project.Software.BMC.Updater`, 而这里的 BIOS 更新 (`bios-update.sh`、`obmc-flash-host-bios@.service`) 建立在经典流程上, 所以切换需要重做并在板上测试 BIOS 更新。
- 以上都没有构建和运行验证。

RTC 与日志时间

- NCT3015Y-R 接在 AST2600 的 I2C10 (Linux `i2c-9`, 地址 `0x6f`), 由 `nuvoton,nct3018y` 驱动绑定 (与 NCT3015Y 的寄存器兼容性没有在硬件上验证)。AST2600 内部 RTC 没有电池, 所以在设备树里禁用, NCT3015Y 是 `rtc0`。
- BMC 系统时间以及随之而来的 SEL 和 journal 时间戳默认来自 RTC: 开机时 `CONFIG_RTC_HCTOSYS`, `ceb-gnrd-rtc-sync` 兜底 (最多等 `/dev/rtc0` 3 秒, 所以没有 RTC 的机器要等满 3 秒; 启动超时 5 秒)。NTP 同步后由 `CONFIG_RTC_SYSTOHC` 写回时间。RTC 要保持 UTC。

板级硬件图

由工作簿导出的硬件图安装为 `/usr/share/ceb-gnrd/ceb-gnrd-hardware-contract.yaml`, 记录 16 条 I2C 总线、CPU 的 I3C3 管理总线、AST2600 ADC 引脚 0-15、6 路 PWM/TACH 风扇通道, 以及已命名的电源、复位和告警 GPIO。机箱开盖检测使用 AST2600 专用的 CHASI# 入侵输入, 经入侵 `hwmon` 锁存 (`0002-hwmon-add-AST2600-chassis-intrusion-driver.patch`)。

- I2C7 (Linux `i2c-6`) 上 4 个 NST175H-QSPR 温度传感器: 进风口 `0x48`、出风口 `0x49`、PCIe `0x4a`、M.2 `0x4b`, 只做测量。它们由 `Entity-Manager / dbus-sensors` (Type `LM75A`) 创建, 不在设备树里声明 (两处都声明会让每次扫描都记录 `Failed to register i2c client lm75a ... (-16)`)。
- ADC: 两个 ADC 引擎都使用 2.5 V 内部参考。 `D3V0_BAT0` 按现状读取 (R542/Q39 未焊, 3 V 电池使输入饱和), 直到原理图修正。
- CRPS 电源: 原理图 I2C8 (Linux `i2c-7`), PMBus 地址 `0x58 / 0x59 / 0x5a`, 装 0 到 2 个模块。设备树不声明 PMBus 节点: `ceb-gnrd-psu-detect` 每 5 秒用 `STATUS_BYTE` 读取轮询这三个地址, 为每个模块创建或删除 `pmbus` 设备, 所以空槽不会产生探测失败日志。`Entity-Manager/PSUSensor` 发布输入和输出的电压与功率以及 `PSUn_Temp` (`pmbus` 的 `temp2`)。没有配置在位、冗余或阈值告警。
- CPU PROM/SMBUS_HOST: `BMC_PROM_SCL/SDA` 在 AST2600 的 I2C15 (Linux `i2c-14`), 不创建 EEPROM 客户端。
- 板卡版本: `PCB_VER[2:0]` 作为 GPIO 输入采样, 见本章末尾。

BMC 网口

- `eth0`: MAC2 配 RTL8211FS-CG, 接独立的管理 RJ45, 默认 IPv4 DHCP (地址、网关、DNS 由 DHCP 服务器提供, 仍可在网页或 IPMI 里改成静态)。RGMII 模式 `rgmii` (PHY 不加延时, RXDL strap 关闭), PHY 地址按原理图备注为 2 (strap 电阻读出来是 1, 需上板确认), PHY 复位 `RTL8211_SYS_RSTN` 由 CPLD 驱动。RGMII 延时仍要在板上检查。

- `eth1` : MAC3, NC-SI 接 Intel E810 (IPMI LAN 通道 2) , DHCP。E810 没有待机供电, 所以 `ceb-gnrd-ncsi` 在主机关机时让链路保持关闭, 机箱电源状态变为 On 时把它拉起, 并且因为 E810 可能不会立刻应答, 在没有载波时每 30 秒把链路 down/up 一次, 最多 3 次。systemd-networkd 自己不改这个接口的管理状态 (`ActivationPolicy=manual`) 。
- MAC 地址来自 U-Boot 环境变量 `ethaddr` / `eth1addr` ; 网络管理服务关闭 `persist-mac` 和 `sync-mac` , 不接受通过 D-Bus / Web / Redfish 修改 MAC; `systemd-networkd` 启动前会清掉旧网络配置里的链路 MAC 覆盖 (`ceb-gnrd-clear-network-mac.sh`) , IP、DHCP、DNS 和静态邻居设置保留。

Linux 和 U-Boot 的设备树保持这个对应关系: `phandle mac1` 是物理 MAC2 (RTL8211FS, `mdio1/ethphy1`) , `phandle mac2` 是物理 MAC3 (NCSI3, E810) ; 物理 MAC1 和 MAC4 被禁用。刷入更新后的镜像后应该只出现这两个设备。

告警、SEL 与系统告警灯

- 一个共用的 LED `BMC_SYS_ALERT_LED` (GPIOI5, 内核 LED 名 `fault`) 显示四种告警 (`ceb-gnrd-alert-led`) 。它由 `phosphor-led-manager` 驱动: 服务只置位/清除标准组 `enclosure_fault` (`Asserted` 属性, 告警期间每 30 秒重发一次) , `led.json` 把该组映射到 `fault` LED。电压阈值告警和温度不可恢复上限 (UNR) 告警在告警解除时熄灭。CPU_MAX_TEMP 和 DIMM_MAX_TEMP 到达 Upper Critical (98 °C / 85 °C) 也点亮, 低于阈值且没有其他故障时熄灭; 主机 watchdog 超时和 BIOS 启动失败 (电源正常后 600 秒内 `BMC_BIOS_BOOT_OK` 没有置位) 也点亮。BIOS 启动失败在之后 POST 成功完成时清除 (包括主机复位之后) ; 只有主机 watchdog 超时一直锁存到 BMC 重启。任何一种都会点亮这个灯。服务自己不读任何 GPIO: 主机电源状态来自 `xyz.openbmc_project.State.Chassis` 的 `CurrentPowerState` , BIOS 启动 OK 来自 `OperatingSystemState` (`xyz.openbmc_project.State.OperatingSystem` , `/xyz/openbmc_project/state/host0`) ; x86-power-control 独占这两根线 (`PowerOk` `BMC_CPU_PWRGD` , 标准的 `PostComplete` `BMC_BIOS_BOOT_OK` , 高有效) 。主机开着时 `PostComplete` 的下降沿会启动其状态机的热复位检查并记录一次软复位的重启原因, 不发电源脉冲。没有在板上运行过。
- 四种告警都会写 SEL: 电压和温度由 `phosphor-sel-logger` 的阈值监视器写入 (板级补丁 `0001-ceb-gnrd-log-non-recoverable-threshold-events.patch` 让它把私有的 `NonRecoverable` 接口当作上/下不可恢复处理) , watchdog 由它的 `watchdog` 监视器写入, BIOS 失败由告警服务自己写入。
- rsyslog 加载 `imjournal` (`recipes-extended/rsyslog/rsyslog_%.bbappend`) : 没有它, rsyslog 看不到 `IPMI_SEL_*` 的 journal 字段, `/var/log/ipmi_sel` 一直是空的 (即使 `sel-logger` 记录了事件, SEL 也显示 “没有条目”) 。
- 网页的 “事件日志” 页是 Redfish 事件日志, 与 IPMI SEL 是不同的文件: `bmcweb` 读取 `/var/log/redfish` (行格式 `<时间戳> <MessageId>, <MessageArgs>` , 只有它的注册表里存在的消息 ID 才显示) , rsyslog 把带 `REDFISH_MESSAGE_ID` 的 journal 条目写进去 (`ceb-gnrd-redfish.conf` , 与 Intel 参考平台同样的规则; `sel-logger` 的阈值事件和电源按键消息都带有这个 ID) 。没有这条规则, 即使 `ipmitool sel list` 有记录, 该页也是空的。 `redfish` 与 SEL 在同一次 `logrotate` 里轮转 (64k, 保留 1 个旧文件) 。
- SEL 是用标准 `logrotate` 实现的 rollover 日志 (`ceb-gnrd-sel-logrotate`) : `phosphor-sel-logger` 读取 `/var/log/ipmi_sel*` (所有轮转文件) , 并把下一个记录 ID 保存在自己的文件里, 所以 ID 不会重复使用。一个定时器在开机 30 秒后首次、之后每 1 分钟运行 `logrotate` , `size 15k` 、 `rotate 1` ,

即大约保留最新的 100 到 200 条记录，更老的删除（按大小计算，所以条数是近似值，突发的记录最多可在 1 分钟的检查间隔内超出）。`/var/log` 是否能经过重启保留还没有检查。

- 网页的事件日志通过 sel-logger 的 journal 记录由 SEL 提供数据。

IPMI

- `mc info`：Device ID 32, Device Revision 2, Product ID 3346 (0x0D12), Manufacturer ID 6659 (0x1A03), BMC 上的 ipmitool 显示为 `CTOPAI / CEB-GNR-D` (`dev_id.json`, 见第六章)。板卡版本是第四个 AUX 固件版本字节。固件版本来自 `DISTRO_VERSION`, 定义在厂商发行版 `meta-ctopai/conf/distro/ctopai-openbmc.conf` (`local.conf` 里 `DISTRO = "ctopai-openbmc"`; 2.0.0; 固件版本从 2.0 开始, `ipmitool mc info` 显示为 2.00)。
- 传感器：启用了 `dynamic-sensors` 和 `hybrid-sensors`, 所以所有 D-Bus 传感器 (ADC、温度、风扇、CPU/DIMM 最大值、PSU) 都能在 IPMI 里看到, 与静态的主机状态传感器并列。
- 电压阈值 (标称 $\pm 15\%$) 使用 Critical 级别 (lower critical / upper critical), 这样 ipmitool 能显示; Entity-Manager 里板子名为 “CEB-GNRD” (Redfish 机箱 “CEB_GNRD”)。
- LAN: `phosphor-ipmi-net` 在 `eth0` 上提供 RMCP+。SOL、用户、通道和会话命令使用标准的 `phosphor-host-ipmid` provider。
- DCMI 的电源读数和温度读数没有配置 (`power_reading.json` 没有路径, `dcmi_sensors.json` 为空), 所以这些命令没有有用的返回。
- `ipmitool fru gen [文件]` (对 ipmitool 的板级补丁) 交互式生成带机箱、板卡和产品信息区的 FRU 镜像 (默认 `fru.bin`)：每个提示都显示格式, 默认值已经填在输入行里, 可以修改, 直接回车就保留。用 `ipmitool fru write 0 fru.bin` 写入。FRU 0 是板载 EEPROM (机箱类型 0x17 或 0x11 对应 FRU ID 0); `ceb-gnr-d-fru` 在开机时往空白的 EEPROM 写入默认 FRU, 并在每次写入后让 `fru-device` 重新扫描。
- SSH 是 dropbear (22 端口); 装了 `openssh-sftp-server` 和 `openssh-scp`, 所以 `scp` 在基于 SFTP 的协议和旧协议下都能用。

Web 界面

- 语言：英文 (`en-US`) 和简体中文 (`zh-CN`), 其余语言被过滤, 包括之前保存的选择。`zh-CN.json` 是本层维护的完整翻译。
- 网页在构建时打补丁 (`recipes-phosphor/webui/files`, 登记在 `webui-vue_%.bbappend` 里, 编号 0001–0030)：语言 (0001–0002)、风扇控制页 (0003)、移除 SNMP、清除密钥、LDAP 和资源管理电源页 (0004)、KVM 全屏 (0006)、只保留 BMC 的恢复出厂 (0007)、清单只保留系统/BMC/机箱 (0008)、移除概览页电源卡 (0009)、无备份镜像卡 (0010)、只保留 BMC dump (0011)、去掉虚拟 TPM / RTAD 开关 (0012)、固件更新进度与 BIOS 刷写区域选择 (0013–0014)、传感器分模拟/离散标签 (0015) 和分页 (0017)、POST 码最新在前 (0016、0027)、固件更新进度跨页面保留 (0018)、恢复出厂措辞 (0019)、概览固件卡 (0020)、电源操作和实时页面刷新 (0021、0024)、事件日志的列和排序 (0022、0023、0028)、虚拟媒体会话关闭与读回复分块 (0025、0026)、BMC 更新分区选择与更新完成通知 (0029、0030)。固件页的 BMC 和 BIOS 都没有备份镜像卡 (0010)。
- `bmcweb` 使用 `dbus-rest` (风扇页)、`redfish-dump-log` (转储页; 不加这个选项, 配方会去掉转储路由) 和 `redfish-updateservice-use-dbus=disabled` 构建：默认情况下 Manager 的 `FirmwareVersion` 在 `/xyz/openbmc_project/software/bmc/functional` 下查找, 而这里用的经典

phosphor-image-updater 发布在 `/xyz/openbmc_project/software/functional` (修改后没有验证)。BMC 转储由 `phosphor-debug-collector` 产生, 本平台没有 System dump。此外还加了补丁: 删除单条事件日志而不重排 ID、虚拟媒体异步清理和接收背压、服务根返回 BMC 启动 ID (给更新监视使用)。

- 虚拟媒体: 只支持“从浏览器读取镜像文件”(bmcweb 的 `/vm/0/0` WebSocket、jsnbd、nbd, 经 vHub 的 USB 大容量存储到主机)。外部服务器 (CIFS/HTTPS) 模式需要已停止维护的 virtual-media 服务, 既不编译也不显示。
- 固件页的 BIOS 卡片的运行版本显示 `--`; BIOS 版本报告没有实现。
- 概览页的“电源信息”卡、功率上限以及任何需要 DCMI 电源支持的内容都已移除。

VGA 显示输出与 KVM

- AST2600 GFX DAC 输出提供 CPU 的 VGA 显示通路 (`CONFIG_DRM_ASPEED_GFX`、`&gfx`、`GPIOL6/VGAHS` 和 `GPIOL7/VGAVS`)。DDC 引脚是固定功能。
- 单独的 `CONFIG_VIDEO_ASPEED` / `&video` 通路 (保留内存 `video_engine_memory`) 为 KVM 采集主机视频; 镜像带有 `obmc-ikvm` 和 bmcweb 的 KVM 端点。AST2600 的 USB2A D+/D- 接到主机 VL805 的 USB 端口 4: vHub 工作在设备模式 (`pinctrl_usb2ad_default`), EHCI 主机模式禁用, 由 `configs` HID 提供键盘和鼠标。

主机串口 SOL

- SOL 使用 AST2600 的 VUART1: 主机看到的是 COM1 (I/O `0x3F8`), 经 eSPI Peripheral 通道, 所以是双向的 (设备树里的 `&vuart1`, `CONFIG_SERIAL_8250_ASPEED_VUART`)。 `obmc-console` 使用 `ttvUART0` (来自 `udev-aspeed-vuart` 的软链接, VUART1 在 `0x1E787000`) 作为 `OBMC_CONSOLE_HOST_TTY`, 配置为 `server.ttyVUART0.conf` (默认的 socket 名, bmcweb 和 IPMI SOL 才能连上)。BIOS 的串口重定向必须设为 COM1。
- 为什么用 VUART: BIOS 检测到 AST2600 的 SuperIO, 把 COM1 路由到 eSPI 并一直轮询行状态寄存器 `0x3FD`; 后面没有可用的 COM1 时该寄存器读出 `00` (“发送器非空”), BIOS 就卡死。运行中的 VUART 会正确应答这个寄存器。必须有东西读取 VUART 的数据 (`obmc-console` 会读), 否则缓冲区满了, 寄存器又变回“非空”, BIOS 可能再次卡死。
- UART3 RX (`GPIOL5/RXD3`, 只收) 仍保持复用, 但不再是 SOL 的来源。网页 SOL 页是原版的 (可以输入); 旧的只读补丁 `0005` 已经移除。
- AST2600 的 SuperIO (I/O `0x2E/0x2F`) 保持启用: BIOS 能找到它并把 COM1 路由到 VUART。 (`SCU510[3]` 可以禁用它, 但只有上电复位才能清除这一位, 而且 BIOS 之后可能完全无法使用 COM1。)
- UART5 (`ttys4`, 115200 波特, 球 C8/D8) 是本地的 BMC 调试控制台, U-Boot 和 Linux 都用它。

FRU EEPROM 访问

- **fru-device** (entity-manager) 探测 I2C 上的 FRU EEPROM 并发布到 D-Bus。
- **phosphor-ipmi-fru** 接到原理图 I2C11 (Linux `i2c-10`) 上的 FM24C08D, 地址块 `0x50-0x53`, 1 KiB, 页大小 16 字节。原理图显示 `BMC_FRU_WP` (封装球 D21, GPIOG6) 有下拉, 所以默认允许写入。

- IPMI 的 FRU 读写命令 (0003-ceb-gnrd-fru-area-is-the-whole-eeeprom.patch) 把整颗 EEPROM 当作一个 FRU 区, 兼容 fru-device; fru-device 另有两个补丁: 允许写入与 EEPROM 等大的镜像 (0001-...), 写入后回读校验, 成功才返回成功 (0002-...) 。
- ceb-gnrd-yaml-config.bb 提供 FRU 的 YAML 映射文件 (IPMI_FRU_YAML / IPMI_FRU_PROP_YAML); 实体 ID 和实例都是 0。

待确认事项

下面这些都需要板子 (或者在等你的决定) :

- eSPI Peripheral 通道的启动和 BIOS 的 KCS 端口 (0xCA2) 。
- PHY 地址 2、RGMII 延时, 以及真实板子上的 U-Boot/Linux 网络。
- 主机上电后 NC-SI 链路的时序 (重试间隔和次数都是猜的) 。
- PSU STATUS_BYTE 探测、把 temp2 当作 PSU 温度、PSU 传感器命名。
- CPU 和 DIMM 温度的 PECI hwmon 标签和数值 (由 ceb-gnrd-temp-max 读取并记录日志) 和风扇 PWM 对象名。
- 风扇曲线温度 (占位值) 、IANA 厂商编号 (0x1A03, 按给定值) 、固件版本规则。
- 各种告警的 SEL 记录、SEL rollover、以 RTC 为时间源、SOL 输出和 KVM 在真实硬件上的检查。
- 没有实现: BIOS 版本报告 (sbp1 基于 coreboot 的 bios-version 不适用于 UEFI BIOS) 、基于 SMBIOS 的清单和 DCMI。
- D3V0_BAT0 在 R542/Q39 焊上之后的缩放。

IPMI 里的板卡版本

三个 PCB_VER[2:0] strap 输入作为 GPIO 输入采样。它们的原始逻辑电平把板卡版本编码为 (PCB_VER2 << 2) | (PCB_VER1 << 1) | PCB_VER0 , 得到 0 到 7 的值。 ipmitool mc info 把这个值放在 AUX 固件版本字段的第四个 (最后一个) 字节里, 前三个 AUX 字节不变。 CFG_VER0 是另一个配置 strap, CFG_VER1 保留, 二者都不计入 PCB 版本。

AST2600 是 ASPEED 公司的 ARM 架构服务管理 SoC, 更多信息见[这里](#)。

十、代码目录与文件清单 (meta-ceb-gnrd)

本章列出 meta-ctopai/meta-ceb-gnrd 层下每个目录和文件的用途与目的。路径均相对于该层根目录。命名约定: ceb-gnrd-*.bb 是本板自有的 Yocto 配方, *_.bbappend 是对上游配方的追加, files/ 下是配方引用的源文件, NNNN-*.patch 是对上游源码的补丁 (按编号顺序应用) 。补丁的"目的"一栏取自补丁标题。

10.1 目录总览

目录	用途与目的
conf/	层配置、机器定义、模板配置 (oe-init-build-env 使用) 。
recipes-bsp/u-boot/	U-Boot 板级支持: 设备树、DDR4 参数、网络与环境变量默认值、复位原因传递。

目录	用途与目的
recipes-kernel/linux/	Linux 内核：板级设备树、内核配置片段、AST2600 eSPI/机箱入侵驱动、PECI GNR 支持及若干稳定性补丁。
recipes-core/systemd/	systemd 与网络：eth0/eth1(NC-SI) 网络配置、看门狗、日志与 coredump 限额、启动时清理遗留 MAC。
recipes-extended/rsyslog/	rsyslog：Redfish 事件日志转发与限额配置。
recipes-connectivity/jsnbd/	虚拟媒体 NBD 代理 (jsnbd) 的稳定性修复。
recipes-devtools/qemu/	让 OpenBMC 自己构建的 qemu-system-native 带上本板的 QEMU 板级模型补丁。
recipes-x86/chassis/	x86-power-control：主机上电/复位/强制关机 GPIO 与电源恢复策略。
recipes-phosphor/	本板所有 Phosphor/OpenBMC 应用层定制，按功能分子目录（见下文）。
tools/	开发辅助工具：QEMU 硬件模拟器及其补丁、教程构建脚本。不进入镜像。

recipes-phosphor/ 子目录：

子目录	用途与目的
buttons/	电源/复位/UID 按键的 GPIO 定义及按键日志。
configuration/	硬件契约、entity-manager 板卡配置（传感器、FRU、风扇 PWM）和 FRU/IPMI 的 YAML 映射。
console/	obmc-console：主机串口（VUART）控制台配置。
dump/、logging/	转储与事件日志容量限制（rwfs 仅 10 MiB）。
fans/	风扇控制权移交、风扇设置 D-Bus 服务、CPU/DIMM 最大温度传感器。
flash/	BMC/BIOS 固件更新：分区选择、BIOS 刷写脚本和布局表。
fru/	板载 FRU EEPROM 默认内容初始化与重扫描。
health/	服务重启策略、重启次数限制、看门狗复位日志。
images/、packagegroups/	镜像内容和软件包分组。
initrdscripts/	初始化文件系统：固件更新时跳过 U-Boot、更新白名单。
interfaces/	bmcweb（Web/Redfish 后端）补丁和构建选项。
ipmi/	IPMI 命令集：ipmid 补丁、ipmitool 补丁、传感器/FRU 映射、白名单、风扇 OEM 命令、I2C 白名单。
leds/	告警 LED、eSPI 心跳 LED 及 LED 组定义。
network/	NC-SI 链路管理。
psu/	PSU 在位探测。
rtc/	RTC 与系统时间同步。

子目录	用途与目的
<code>sel-logger/</code>	IPMI SEL 日志及其轮转与清理。
<code>sensors/</code>	dbus-sensors 补丁与配置选择。
<code>settings/</code>	默认设置（来电开机策略、SOL）。
<code>state/</code>	状态管理、启动进度、POST 码历史。
<code>utils/</code>	自检与硬件转储工具。
<code>watchdog/</code>	主机看门狗动作 systemd 单元。
<code>webui/</code>	Web 界面（webui-vue）补丁、中文语言包。

10.2 `conf/`

文件	用途与目的
<code>conf/layer.conf</code>	层注册：层名、优先级、依赖层、 <code>BBFILES</code> 匹配规则。
<code>conf/machine/ceb-gnrd.conf</code>	机器定义：AST2600 SoC、内核设备树名、U-Boot 配置、Flash 分区与镜像布局、包含的功能（PECI、eSPI 等）。
<code>conf/templates/default/bblayers.conf.sample</code>	新建 build 目录时的默认层列表。
<code>conf/templates/default/local.conf.sample</code>	新建 build 目录时的默认 <code>local.conf</code> （机器、发行版）。
<code>conf/templates/default/conf-notes.txt</code>	<code>oe-init-build-env</code> 后在终端显示的构建提示。

10.3 `recipes-bsp/u-boot/`

文件	用途与目的
<code>u-boot-aspeed-sdk_%.bbappend</code>	把本板的补丁、设备树、配置片段加入 U-Boot 构建。
<code>files/ast2600-ceb-gnrd.dts</code>	U-Boot 使用的板级设备树（Flash、网络 PHY、串口等）。
<code>files/ceb-gnrd-ddr4.cfg</code>	DDR4 内存相关的 U-Boot 配置。
<code>files/ceb-gnrd-network.cfg</code>	U-Boot 网络驱动与命令的配置开关。
<code>files/ceb-gnrd-env.h</code>	默认环境变量：默认 MAC、 <code>netupdate</code> （TFTP 更新 kernel/rofs 的命令）。
<code>files/0001-ceb-gnrd-board-device-tree-network-and-environment.patch</code>	加入板级设备树、默认网络和环境变量。
<code>files/0002-ceb-gnrd-pass-boot-reset-cause-to-linux.patch</code>	把原始复位原因（上电/看门狗/外部）传给 Linux，供复位日志使用。

文件	用途与目的
files/0003-ceb-gnrd-fill-missing-default-mac.patch	已保存的环境里缺 MAC 变量时，在网卡探测前补上默认值。

10.4 recipes-kernel/linux/

文件	用途与目的
linux-aspeed_%.bbappend	把配置片段、设备树和下列补丁加入内核构建，并在 do_configure 里安装板级 dts。
files/aspeed-ceb-gnrd.dts	本板内核设备树：GPIO 命名、I2C 总线设备、PWM/风扇、PECI、eSPI、NC-SI 网络、Flash 分区。
files/espi-peci.cfg	内核配置片段：启用 PEGI、eSPI 及相关 hwmon 选项。
files/0001-soc-aspeed-add-AST2600-eSPI-peripheral-ready-driver.patch	新增 AST2600 eSPI 外设通道使能驱动（主机侧 I/O 就绪）。
files/0002-hwmon-add-AST2600-chassis-intrusion-driver.patch	新增 AST2600 机箱入侵锁存 hwmon 驱动。
files/0003-peci-add-Granite-Rapids-CPU-and-DIMM-temperature.patch	为 peci-cputemp/dimmtemp 增加 GNR/GNR-D（表按 EMR 抄写，待实机验证）。
files/0004-pmbus-ratelimit-optional-device-probe-message.patch	PMBus 状态寄存器不可读的警告限速，避免刷屏。
files/0005-usb-gadget-hid-classify-endpoint-shutdown.patch	USB HID gadget 端点关闭时的取消请求不再按错误处理。
files/0006-ncsi-accept-initial-deselect-response.patch	NC-SI 发现阶段在 package 注册前接受 deselect 响应。

10.5 recipes-core/、 recipes-extended/、 recipes-connectivity/、 recipes-devtools/

文件	用途与目的
recipes-core/systemd/systemd_%.bbappend、 systemd-conf_%.bbappend	安装下列 systemd 配置片段与网络文件。
recipes-core/systemd/files/10-ceb-gnrd-eth0.network	eth0 (RJ45, MAC2/RGMII) 网络配置，默认 IPv4 DHCP。
recipes-core/systemd/files/20-ceb-gnrd-eth1-ncsi.network	eth1 (NC-SI 共享网口) 网络配置。
recipes-core/systemd/files/20-ceb-gnrd-boot-mac.conf	networkd 启动前执行清理脚本（drop-in）。
recipes-core/systemd/files/ceb-gnrd-clear-network-mac.sh	清除旧配置里的链路 MAC 覆盖，保留 IP/DHCP/DNS 设置，使 MAC 以 U-Boot 环境为准。

文件	用途与目的
<code>recipes-core/systemd/files/40-hardware-watchdog.conf</code>	让 systemd 喂硬件看门狗，系统卡死时自动复位 BMC。
<code>recipes-core/systemd/files/50-ceb-gnrd-console.conf</code>	sysctl: 常规网络/I2C 内核消息只进 ring buffer/journal, 不与串口登录输入交错（错误仍显示， <code>dmesg -n 8</code> 可恢复）。
<code>recipes-core/systemd/files/60-ceb-gnrd-coredump-limits.conf</code>	限制 coredump 大小（rwfs 只有 10 MiB）。
<code>recipes-core/systemd/files/60-ceb-gnrd-journal-limits.conf</code>	限制 journal 占用空间。
<code>recipes-extended/rsyslog/rsyslog_%.bbappend</code>	安装 rsyslog 配置。
<code>recipes-extended/rsyslog/files/ceb-gnrd-redfish.conf</code>	把日志转为 Redfish 事件日志格式。
<code>recipes-extended/rsyslog/files/rsyslog-override.conf</code>	rsyslog 服务的 drop-in 覆盖。
<code>recipes-connectivity/jsnbd/jsnbd_%.bbappend</code>	应用 nbd-proxy 补丁并安装状态钩子。
<code>recipes-connectivity/jsnbd/files/state_hook</code>	虚拟媒体状态变化时的钩子脚本。
<code>recipes-connectivity/jsnbd/files/0001-stop-reaping-on-echild-and-serialize-gadget-cleanup.patch</code>	处理 ECHILD，并保证 gadget 配置完成后再拆除，避免竞态。
<code>recipes-connectivity/jsnbd/files/0002-nbd-client-explicit-default-export.patch</code>	适配 NBD 3.27，显式指定默认 export。
<code>recipes-devtools/qemu/qemu-system-native_%.bbappend</code>	把 <code>tools/qemu/patches</code> 的板级模型补丁加入 OpenBMC 自己构建的 QEMU。

10.6 recipes-x86/chassis/（主机电源控制）

文件	用途与目的
<code>x86-power-control_%.bbappend</code>	应用补丁并安装 <code>power-config-host0.json</code> 。
<code>files/power-config-host0.json</code>	电源/复位按钮、PowerOK、SIO 等 GPIO 线名与极性、超时参数。
<code>files/0001-ceb-gnrd-add-force-power-button-off-method.patch</code>	提供主机已关机时的强制关机脉冲方法（主机在 S5 收不到 PowerOK 时用）。
<code>files/0002-ceb-gnrd-own-bus-name-for-the-exported-buttons.patch</code>	按键对象使用独立 bus name，避免与 buttons 守护进程冲突。
<code>files/0003-ceb-gnrd-apply-power-restore-only-on-ac-boot.patch</code>	来电恢复策略只在 AC 上电启动时执行，BMC 单独复位不触发开/关机。

10.7 recipes-phosphor/buttons/、 console/、 dump/、 logging/、 images/、 packagegroups/、 initrdscripts/

文件	用途与目的
buttons/obmc-phosphor-buttons_%.bbappend	安装按键 GPIO 定义。
buttons/files/gpio_defs.json	电源键、复位键、UID 键的 GPIO 线名映射。
buttons/ceb-gnrd-power-button-log.bb + files/ceb-gnrd-power-button-log.{sh,service}	记录电源键按下/释放到日志，便于排查。
buttons/files/10-ceb-gnrd-wait-buttons.conf、ceb-gnrd-wait-buttons.sh	等按键 D-Bus 对象就绪后再启动 button-handler，避免竞态导致 UID 键失效。
console/obmc-console_%.bbappend、files/server.ttyVUART0.conf	把主机串口（VUART0）作为 SOL 控制台。
dump/phosphor-debug-collector_%.bbappend	限制 BMC 转储的单个和总大小（rwfs 容量）。
logging/phosphor-logging_%.bbappend	事件日志条数上限设为 64。
images/obmc-phosphor-image.bbappend	让静态镜像打包等待 fitImage 部署，避免 sstate 清理后缺 image-kernel。
packagegroups/packagegroup-ceb-gnrd-apps.bb	本板应用软件包组。
packagegroups/packagegroup-obmc-apps.bbappend	按 Intel 服务器板的需要调整 OpenBMC 应用包组（entity-manager、dbus-sensors、POST 码、BIOS 更新、状态管理等）。
initrdscripts/obmc-phosphor-initfs.bbappend	向初始化文件系统安装下面两个文件。
initrdscripts/files/ceb-gnrd-update-skip-u-boot.sh	固件更新时不覆盖 U-Boot 区。
initrdscripts/files/ceb-gnrd-whitelist	更新期间需要保留的文件/分区白名单。

10.8 recipes-phosphor/configuration/（配置与硬件契约）

文件	用途与目的
entity-manager_%.bbappend	安装 ceb-gnrd.json 并应用 entity-manager 补丁。
entity-manager/ceb-gnrd.json	板卡描述：温度传感器、ADC 电压、PSU、风扇 PWM/转速、FRU EEPROM、板级 Inventory。
entity-manager/0001-ceb-gnrd-fru-device-allow-eeeprom-sized-write.patch	fru-device 允许写入与 EEPROM 等大的 FRU 镜像。
entity-manager/0002-ceb-gnrd-verify-fru-eeeprom-write.patch	写 EEPROM 后回读校验，成功才返回成功。

文件	用途与目的
entity-manager/0003-ceb-gnrd-skip-static-platform-inventory-events.patch	跳过无法识别的静态平台 Inventory 事件，避免噪声。
ceb-gnrd-hardware-contract.bb 、 files/ceb-gnrd-hardware-contract.yaml	硬件契约：把原理图里的信号、地址、极性固化为文本，作为代码与板卡之间的对照依据。
ceb-gnrd-yaml-config.bb	安装 phosphor-ipmi-host 使用的 YAML 映射文件。
ceb-gnrd-yaml-config/ceb-gnrd-ipmi-fru.yaml	IPMI FRU 到 D-Bus 对象的映射。
ceb-gnrd-yaml-config/ceb-gnrd-ipmi-fru-properties.yaml	FRU 字段到 Inventory 属性的映射。

10.9 recipes-phosphor/fans/

文件	用途与目的
ceb-gnrd-fan-services.bb	风扇控制权移交服务（owner/release）。
files/ceb-gnrd-fan-owner.service 、 ceb-gnrd-fan-owner.sh	开机后由 BMC 接管风扇：持有 BMC_FAN_BMC_OVERRIDE_N 并清除其 reset tolerance，确保 BMC 任何复位期间控制权自动回 CPLD。
files/ceb-gnrd-fan-release.sh	停止服务时释放风扇控制权给 CPLD。
ceb-gnrd-fan-settings.bb 、 files/ceb-gnrd-fan-settings.{py,service} 、 com.ctopai.CebGnrd.FanSettings.conf	风扇设置 D-Bus 服务（ApplyFan*），Web 风扇页通过它调速；.conf 是 D-Bus 访问策略。
ceb-gnrd-temp-max.bb 、 files/ceb-gnrd-temp-max.{py,service} 、 com.ctopai.CebGnrd.TempMax.conf	读取 PECI hwmon，合成 CPU_MAX_TEMP / DIMM_MAX_TEMP 传感器（含阈值），Web 与 ipmitool 只显示这两个最大值。
files/10-ceb-gnrd-temp-max.conf	temp-max 服务的启动顺序/依赖 drop-in。

10.10 recipes-phosphor/flash/ 、 fru/ 、 health/

文件	用途与目的
flash/phosphor-software-manager_%.bbappend	应用固件更新补丁，安装 BIOS 刷写脚本。
flash/phosphor-software-manager/0001-ceb-gnrd-bmc-update-partition-selection.patch	固件更新只暂存所选的 BMC 分区。
flash/phosphor-software-manager/bios-update.sh	BIOS 升级：切换 BMC_BIOS_FLASH_SELECT、刷写、恢复（仅升级期间改变该引脚）。
flash/phosphor-software-manager/bios-layout.txt	BIOS Flash 区域布局表。
flash/phosphor-software-manager/obmc-flash-host-bios@.service	调用 bios-update.sh 的 systemd 模板单元。
fru/ceb-gnrd-fru.bb	安装 FRU 初始化与重扫描。

文件	用途与目的
fru/files/ceb-gnrd-fru-init.{sh,service}	EEPROM 空白时写入默认 FRU (CTOPAI / CEB-GNR-D / 93-XXXX-XX) 。
fru/files/ceb-gnrd-fru-rescan.{sh,service}	写入 FRU 3 秒后让 fru-device 重扫，使新内容立即可见。
fru/files/default-fru.bin	默认 FRU 镜像 (144 字节) 。
health/ceb-gnrd-health.bb	安装健康保护相关文件。
health/files/10-ceb-gnrd-restart*.conf	关键服务异常退出后的自动重启策略 drop-in。
health/files/10-ceb-gnrd-reboot-limit.conf 、 ceb-gnrd-quiesce-reboot-limit.sh	限制连续自动重启次数，达到上限进入静默状态，避免无限重启循环。
health/files/ceb-gnrd-quiesce-reboot-clear.{service,timer}	稳定运行一段时间后清除重启计数。
health/files/ceb-gnrd-wdt-reset-log.{sh,service}	开机时根据复位原因记录看门狗复位日志。

10.11 recipes-phosphor/interfaces/ (bmcweb)

文件	用途与目的
bmcweb_%.bbappend	打开 dbus-rest 与 Dump 日志服务，关闭 DBus 更新路径以适配经典 image-updater，上传体积上限 80 MiB，应用下列补丁。
files/0001-ceb-gnrd-delete-single-file-event-log.patch	支持删除单条事件日志而不重排 ID。
files/0002-ceb-gnrd-async-virtual-media-proxy-cleanup.patch	虚拟媒体回收 nbd-proxy 不再阻塞 Web 事件循环。
files/0003-ceb-gnrd-virtual-media-receive-backpressure.patch	虚拟媒体先排空代理写入再读新消息（背压）。
files/0004-ceb-gnrd-report-boot-id-for-update-monitor.patch	ServiceRoot 返回 BMC 启动 ID，Web 据此识别更新后重启。

10.12 recipes-phosphor/ipmi/

文件	用途与目的
phosphor-ipmi-host_%.bbappend	应用 ipmid 补丁。
phosphor-ipmi-config.bbappend 、 phosphor-ipmi-config/dev_id.json	Get Device ID 返回内容 (ID 32、版本、厂商 6659、产品 3346) 。
phosphor-ipmi-fru_%.bbappend 、 phosphor-ipmi-fru/obmc/eproms/system/chassis/motherboard	旧 FRU 提供者的 EEPROM 路径映射 (主板 FRU) 。
ipmitool_%.bbappend	应用 ipmitool 补丁。

文件	用途与目的
files/0001-ceb-gnrd-report-board-revision-in-mc-info.patch	mc info 的 AUX 最后一字节报告 PCB 版本。
files/0002-ceb-gnrd-show-upper-non-recoverable-threshold.patch	传感器列表显示 UNR 阈值。
files/0003-ceb-gnrd-fru-area-is-the-whole-EEPROM.patch	ipmid FRU 命令以整颗 EEPROM 为 FRU 区，兼容 fru-device。
files/0001-ipmitool-fru-add-gen-command.patch	ipmitool fru gen 交互式生成 FRU（带默认值，可编辑）。
files/0002-ipmitool-add-ceb-gnrd-product-name.patch	ipmitool 识别 CEB-GNR-D 产品名。
files/10-ceb-gnrd-wait-sensors.conf	ipmid 等传感器服务就绪后再启动。
ceb-gnrd-ipmi-sensors/config.yaml	IPMI 传感器号/类型/缩放到 D-Bus 传感器路径的映射。
ceb-gnrd-ipmi-sensor-inventory-native.bb	把传感器映射转成 ipmid 使用的 inventory 数据（native）。
ceb-gnrd-ipmi-fru-read-inventory-native.bb	旧的 FRU 读取清单；FRU 现已走 fru-device 路径，可能不再使用。
ceb-gnrd-ipmi-whitelist-native.bb、files/ceb-gnrd-ipmi-whitelist.conf	IPMI 命令白名单，只放行本板需要的 netfn/cmd。
ceb-gnrd-ipmi-fan.bb、files/ceb-gnrd-ipmi-fan/{fan_oem.cpp,meson.build}	风扇 OEM IPMI 命令实现。
ceb-gnrd-ipmi-i2c.bb、files/generate_i2c_allowlist.py	生成并安装 Master Write-Read 命令可访问的 I2C 总线/地址白名单。

10.13 recipes-phosphor/leds/、network/、psu/、rtc/

文件	用途与目的
leds/phosphor-led-manager_%.bbappend、ceb-gnrd-led-manager-config-native.bb、files/led.json	LED 组定义（告警、状态）。
leds/ceb-gnrd-alert-led.bb、files/ceb-gnrd-alert-led.{py,service}	根据传感器告警/严重事件驱动告警 LED。
leds/ceb-gnrd-espi-heartbeat.bb、files/ceb-gnrd-espi-heartbeat.service	eSPI 通信心跳指示。
leds/files/wait-for-espi-driver.sh	等 eSPI 驱动就绪再启动心跳。
network/phosphor-network_%.bbappend	网络默认配置。
network/ceb-gnrd-ncsi.bb、files/ceb-gnrd-ncsi.service、manage-ncsi-link.sh	主机上电后管理 NC-SI 链路（重试间隔与次数待实机确认）。

文件	用途与目的
psu/ceb-gnrd-psu-detect.bb 、 files/ceb-gnrd-psu-detect.{sh,service}	探测 PSU 在位并通知 entity-manager。
rtc/ceb-gnrd-rtc-sync.bb 、 files/ceb-gnrd-rtc-sync.service	以 RTC (NCT3015Y) 为时间源，与系统时间同步。

10.14 recipes-phosphor/sel-logger/ 、 sensors/ 、 settings/ 、 state/ 、 utils/ 、 watchdog/

文件	用途与目的
sel-logger/phosphor-sel-logger_%.bbappend	应用 SEL 补丁，安装日志轮转。
sel-logger/files/0001-ceb-gnrd-log-non-recoverable-threshold-events.patch	记录 UNR/LNR 阈值事件到 SEL。
sel-logger/ceb-gnrd-sel-logrotate.bb 、 files/ceb-gnrd-ipmi-sel.logrotate 、 ceb-gnrd-sel-logrotate.{service,timer}	SEL 日志按计划轮转，控制 rwfs 占用。
sel-logger/files/ceb-gnrd-log-storage-cleanup.{sh,service}	迁移旧版 core/journal 预算，清理遗留 core 文件。
sensors/dbus-sensors_%.bbappend	去掉 intelcpusensor (CPU 温度改由 temp-max 提供)，应用补丁。
sensors/files/0001-reuse-i2c-device-by-config-path-and-guard-psu-io.patch	按配置路径复用 I2C 设备，并保护 PSU I/O。
settings/phosphor-settings-manager_%.bbappend 、 phosphor-settings-manager/settings.override.yml	默认设置：来电开机 (AlwaysOn, 延时 0)、SOL 使能。
state/phosphor-state-manager_%.bbappend	开启 BMC 进入 Quiesced 后自动重启 (次数由 health 限制)。
state/ceb-gnrd-boot-progress.bb 、 files/ceb-gnrd-boot-progress.{py,service}	根据 POST 码/eSPI 状态更新 BootProgress。
state/phosphor-post-code-manager_%.bbappend	POST 码管理定制，应用补丁与限额。
state/files/0001-ceb-gnrd-post-history-warm-boot-cycle.patch	POST 码保留全部取值，并用主机事件区分启动轮次。
state/files/60-ceb-gnrd-post-history.conf 、 ceb-gnrd-post-history-limit.py	将旧的 100 槽 POST 环缩为 2 槽 (rwfs 容量)。
utils/ceb-gnrd-check.bb 、 files/ceb-gnrd-check.{py,sh}	有界的固件自检 (元数据检查，不代表端到端成功)。
utils/files/bmc-hw-dump.sh	只读转储 BMC 硬件使用情况，用于对比厂商固件与新固件。
watchdog/phosphor-watchdog_%.bbappend 、 phosphor-watchdog/*.service	主机看门狗超时动作 (复位/关机/循环) 对应的 systemd 单元。

10.15 recipes-phosphor/webui/

webui-vue_%.bbappend 按编号顺序注册所有补丁； files/zh-CN.json 为简体中文语言包。

补丁	目的
0001 limit-webui-languages	界面语言只保留英文和中文。
0002 add-simplified-chinese-locale	新增简体中文语言。
0003 add-fan-control-page	新增风扇控制页。
0004 remove-resource-management-power	移除 SNMP Alerts、Key clear、LDAP 和资源管理。
0006 kvm-full-screen	KVM 增加全屏按钮。
0007 factory-reset-bmc-only	恢复出厂只提供 BMC 复位。
0008 inventory-supported-tables-only	清单页只保留有后端数据的表。
0009 remove-overview-power-card	概览页移除电源信息卡。
0010 firmware-single-bank	固件页移除备份镜像卡片。
0011 dumps-bmc-only	转储页只保留 BMC 转储。
0012 policies-remove-vtpm-rtad	策略页移除 vTPM、RTAD 开关。
0013 firmware-update-progress	固件更新进度与 BIOS 刷写区域选择。
0014 firmware-cards-side-by-side	BMC 与 BIOS 卡片并排，更新表单加宽。
0015 sensors-discrete-table	传感器分模拟/离散两个标签页。
0016 post-codes-newest-first	POST 码最新在前。
0017 sensors-pagination	模拟传感器表分页。
0018 firmware-progress-survives-page-change	离开固件页后回来仍保留更新进度。
0019 factory-reset-bmc-wording	恢复出厂页去掉服务器相关措辞与关机警告。
0020 overview-firmware-card	概览固件卡只显示运行版本，去掉备份与系统固件版本。
0021 refresh-server-power-operation-state	等待电源操作时刷新电源状态。
0022 event-log-explicit-columns	事件日志移除不支持的状态列、避免空字段。
0023 event-log-actions-heading	事件日志操作列加标题。
0024 refresh-live-status-pages	传感器等实时页面自动刷新。
0025 close-virtual-media-websocket-on-stop	停止时关闭虚拟媒体会话并取消挂起读取。
0026 chunk-virtual-media-read-replies	NBD 回复拆成有界 WebSocket 消息。
0027 post-code-table-sort-api	POST 码表改用当前排序 API。
0028 event-logs-newest-first	事件日志默认最新在前。
0029 bmc-update-partition-selection	BMC 分区选择，并需引导程序显式确认。
0030 bmc-update-completion-notification	识别 BMC 重启并保留更新完成通知。

编号 0005 当前没有对应文件。

10.16 tools/ (开发辅助, 不进入镜像)

文件	用途与目的
tools/expand-to-tutorial-build.py	把本层、父层配置和 quick-start.md 展开到 .tutorial-build/ceb-gnrd/ (git 忽略的临时目录), 用于教程构建。
tools/qemu/README.md	QEMU 模拟器使用说明与补丁清单。
tools/qemu/run-qemu.sh	启动 ceb-gnrd 镜像 (ast2600-evb), 挂载各 I2C 器件、双 Flash、网络并打开浏览器控制面板 (http://localhost:8800)。
tools/qemu/build-qemu.sh	没有 Yocto 环境时, 单独构建带补丁的 QEMU。
tools/qemu/host-sim.py	模拟主机的控制台/浏览器面板后端, 显示 BMC 与主机间信号。
tools/qemu/host_io.py	主机侧 eSPI 传统 I/O 与 USB 枚举的模拟。
tools/qemu/panel.html	模拟器浏览器控制面板的页面。
tools/qemu/panel_service.py	面板的 VGA 画面选择与虚拟媒体 USB 检查。
tools/qemu/log-sink.py	排空模拟器日志 FIFO, 保留当前和一份备份各 8 MiB。
tools/qemu/diagnose-bmc.sh、diagnose-web.sh、diagnose-virtual-media.sh	只读诊断脚本 (固件整体、Web 后端、虚拟媒体挂载失败后)。
tools/qemu/kvm/{os,post,test-pattern}.jpg	模拟 VGA 的静态画面 (系统、POST、测试图)。
tools/qemu/EVENTLOG-DELETE-FIX.md、FAN-OWNER-GLOB-FIX.md、GUI-HEARTBEAT-RESET.md、VUART-STARTUP-FIX.md	模拟器相关问题的修复记录。
tools/qemu/patches/0001-0026	QEMU 板级模型补丁: SCU 时钟、PWM 风扇转速、bmc-host-sim 模拟主机、ADC 可设电压、GPIO 电平保持与 reset tolerance、CRPS PSU、PECI 响应与 GNR-D 温度、NCT3015Y RTC、VUART 与 80h POST 码、视频引擎、eSPI/USB/机箱集成、复位原因保留、AT24C 大小处理、ftgmac100 NC-SI 配置。编号有两个 0020 (ADC 位宽、强制关机脉冲), 按文件名顺序应用。